Von Gamification zu Game-Based Learning

Konzeption und Implementierung eines mobilen Lernspiels für den Mehrtaktsprozessor
Bachelorarbeit im Fachbereich Informatik von Artur Galyautdinov
Tag der Einreichung: 13. April 2026

1. Gutachten: Dr. rer. nat. Svana Esche
2. Gutachten: Prof. Dr-Ing. Andreas Koch
Darmstadt



TECHNISCHE UNIVERSITÄT DARMSTADT

Fachbereich Informatik
Fachdidaktik Informatik

Erklärung zur Abschlussarbeit gemäß § 22 Abs. 7 APB TU Darmstadt

Hiermit erkläre ich, Artur Galyautdinov, dass ich die vorliegende Arbeit gemäß § 22 Abs. 7 APB TU Darmstadt selbstständig, ohne Hilfe Dritter und nur mit den angegebenen Quellen und Hilfsmitteln angefertigt habe. Ich habe mit Ausnahme der zitierten Literatur und anderer in der Arbeit genannter Quellen keine fremden Hilfsmittel benutzt. Die von mir bei der Anfertigung dieser wissenschaftlichen Arbeit wörtlich oder inhaltlich benutzte Literatur und alle anderen Quellen habe ich im Text deutlich gekennzeichnet und gesondert aufgeführt. Dies gilt auch für Quellen oder Hilfsmittel aus dem Internet.

Diese Arbeit hat in gleicher oder ähnlicher Form noch keiner Prüfungsbehörde vorgelegen.

Mir ist bekannt, dass im Falle eines Plagiats (§38 Abs.2 APB) ein Täuschungsversuch vorliegt, der dazu führt, dass die Arbeit mit 5,0 bewertet und damit ein Prüfungsversuch verbraucht wird. Abschlussarbeiten dürfen nur einmal wiederholt werden.

English translation for information purposes only:

Thesis Statement pursuant to § 22 paragraph 7 of APB TU Darmstadt

I herewith formally declare that I, Artur Galyautdinov, have written the submitted thesis independently pursuant to § 22 paragraph 7 of APB TU Darmstadt without any outside support and using only the quoted literature and other sources. I did not use any outside support except for the quoted literature and other sources mentioned in the paper. I have clearly marked and separately listed in the text the literature used literally or in terms of content and all other sources I used for the preparation of this academic work. This also applies to sources or aids from the Internet.

This thesis has not been handed in or published before in the same or similar form.

I am aware, that in case of an attempt at deception based on plagiarism (§38 Abs. 2 APB), the thesis would be graded with 5,0 and counted as one failed examination attempt. The thesis may only be repeated once.

Datum / Date: Unterschrift/Signature:

13.04.2026_____ _____

Zusammenfassung

Die vorliegende Bachelorarbeit beschäftigt sich mit der Konzeption und Implementierung eines mobilen Lernspiels zur Vermittlung der Architektur des Mehrtaktprozessors. Ausgangspunkt ist die Beobachtung, dass Studierende des Moduls „Rechnerorganisation“ an der Technischen Universität Darmstadt dazu neigen, spezifische Lösungsmuster auswendig zu lernen, anstatt ein tiefgreifendes Verständnis der zugrundeliegenden Hardware-Logik zu entwickeln. Um dieser Problematik entgegenzuwirken, wird ein Ansatz des Game-Based Learning verfolgt, bei dem die Architektur des Mehrtaktprozessors nicht bloßer Lerngegenstand, sondern das Kernelement der Spielmechanik ist.

Das entwickelte Spiel führt die Nutzenden entlang eines didaktisch strukturierten Lernpfads – von grundlegenden Digitaltechnik-Komponenten wie Registern, Multiplexern und der ALU bis hin zur vollständigen Simulation des Mehrtaktprozessors auf Basis der RISC-V-Architektur. Das Design stützt sich auf etablierte pädagogische Konzepte, darunter die Cognitive Load Theory, das Scaffolding-Prinzip sowie die Flow-Theorie, um eine motivierende und kognitiv effiziente Lernerfahrung zu gewährleisten.

Die technische Umsetzung erfolgte in der Unity-Engine und zielt primär auf Android-Endgeräte ab. Zur Sicherstellung einer stabilen Performance auf mobilen Geräten wurden umfangreiche grafische und rechnerische Optimierungsmaßnahmen implementiert. Eine explorative Nutzerstudie mit 12 Probanden ergab einen SUS-Score von 80 Punkten, was die Anwendung im Bereich „Excellent“ einordnet, und bestätigte das didaktische Potenzial des Ansatzes. Die Anwendung ist kostenfrei über den Google Play Store sowie itch.io verfügbar.

Hinweis zur Verwendung von KI

Bei der Erstellung der vorliegenden Arbeit wurden Werkzeuge der künstlichen Intelligenz unterstützend eingesetzt.

Das Modell Gemini (Google) wurde primär für die Übersetzung von Textpassagen ins Deutsche, die Korrektur der Grammatik sowie zur Optimierung der Satzstruktur und Terminologie verwendet. Zudem leistete das Modell Hilfestellung bei der technischen Formatierung des Dokuments in LaTeX. Ergänzend wurde das Modell Claude (Anthropic) herangezogen, um die kontextuelle Korrektheit der verwendeten Fachbegriffe zu prüfen.

Außerden wurde Gemini bei der Erstellung von Quellcode-Kommentaren eingesetzt, um die Verständlichkeit und Einheitlichkeit der Code-Dokumentation zu verbessern.

Alle von den KI-Modellen generierten Vorschläge wurden kritisch geprüft, manuell überarbeitet und in den Gesamtkontext des Inhalts integriert.

Inhaltsverzeichnis

1 Einleitung **7**
- 1.1 Motivation und Relevanz . . . 7
- 1.2 Problemstellung . . . 9
- 1.3 Zielsetzung der Arbeit . . . 10
- 1.4 Rahmen der Arbeit . . . 11
- 1.5 Aufbau der Arbeit . . . 12

2 Theoretische Grundlagen **14**
- 2.1 Grundlagen der Rechnerarchitektur . . . 14
  - 2.1.1 Grundlagen der Digitaltechnik . . . 14
  - 2.1.2 Aufbau des Mehrtaktprozessors . . . 15
  - 2.1.3 Komponenten des Datenpfads . . . 18
  - 2.1.4 Die Steuereinheit und das Prinzip der Finite State Machine . . . 19
  - 2.1.5 RISC-V Instruktionsarchitektur . . . 19
- 2.2 Pädagogische Grundlagen . . . 20
  - 2.2.1 Gamification und Game-Based Learning . . . 20
  - 2.2.2 Flow-Theorie . . . 21
  - 2.2.3 Cognitive Load Theory nach Sweller . . . 22
  - 2.2.4 Scaffolding als didaktisches Prinzip . . . 22

3 Marktanalyse und Abgrenzung **24**
- 3.1 Spielerische Hardware-Simulationen (Desktop) . . . 24
- 3.2 Akademische Simulationswerkzeuge (Desktop) . . . 25
- 3.3 Mobiles Segment . . . 26
- 3.4 Abgrenzung des Projekts . . . 26

4 Konzeption und Design **28**
- 4.1 Game Design Document . . . 28
- 4.2 Didaktisches Level-Design . . . 29

4.3 Visueller Stil und Ästhetik . . . 31
4.4 Mobile UI/UX Design . . . 34

5 Implementierung 36
5.1 Technisches Grundkonzept . . . 36
5.1.1 Technischer Stack . . . 36
5.1.2 Softwarearchitektur . . . 37
5.1.3 Szenen- und Ressourcenmanagement . . . 40
5.1.4 Plattformspezifische Unterschiede . . . 43
5.2 Datenmanagement mittels Scriptable Objects . . . 45
5.2.1 Konfiguration der Prozessor-Level . . . 45
5.2.2 Struktur des Dialogsystems . . . 47
5.2.3 Verwaltung von Metadaten und Zuständen . . . 48
5.3 Simulation der Schaltungen . . . 49
5.3.1 Taktlogik und Zustandsverwaltung . . . 50
5.3.2 Signalfluss und Komponentenkommunikation . . . 52
5.3.3 Validierung der Spielerlösung . . . 53
5.4 Distribution und Deployment . . . 54

6 Optimierung 57
6.1 Grafische Optimierung . . . 58
6.1.1 Schatten und Beleuchtung . . . 59
6.1.2 Kamera und Objekte auf der Szene . . . 59
6.1.3 Objektoptimierung . . . 62
6.1.4 Optimierung der Benutzeroberfläche . . . 63
6.2 Rechenoptimierung . . . 68
6.2.1 Event-Driven Design . . . 68
6.2.2 Optimierung des Objektlebenszyklus durch Coroutines . . . 68
6.2.3 Effiziente Datenverwaltung: Scriptable Objects . . . 69
6.2.4 Animationsoptimierung mit DOTween . . . 70

7 Evaluation und Testing 72
7.1 Methodik und Durchführung . . . 72
7.1.1 Methodik und Testaufbau . . . 72
7.1.2 Durchführung der Nutzertests . . . 73
7.2 Analyse . . . 74
7.2.1 Quantitative Auswertung der Bedarfsanalyse . . . 74
7.2.2 Quantitative Auswertung der Pilotierung . . . 76

8 Fazit und Ausblick **78**
8.1 Zusammenfassung der Ergebnisse 78
8.2 Zukünftige Erweiterungsmöglichkeiten 79

1 Einleitung

1.1 Motivation und Relevanz

Die Vermittlung von Inhalten im Bereich der Technischen Informatik, insbesondere der Rechnerorganisation, steht vor einer zentralen Herausforderung: der Abstraktion komplexer, dynamischer Vorgänge.

Das Modul „Rechnerorganisation“ — eine Pflichtveranstaltung im zweiten Semester des Bachelorstudiengangs Informatik an der TU Darmstadt — vermittelt Studierenden die Architektur von Mikroprozessoren auf Basis der RISC-V-Architektur, die Grundlagen der Assemblerprogrammierung sowie die Organisation von Speicher- und Ein-/Ausgabesystemen. Es baut dabei auf dem Pflichtmodul „Digitaltechnik“ aus dem ersten Semester auf, in dem Studierende die Grundlagen digitaler Schaltungen kennenlernen: von der booleschen Algebra und logischen Gattern über kombinatorische und sequenzielle Schaltkreise bis hin zu Hardware-Beschreibungssprachen. Diese Abhängigkeit zwischen den Modulen ist kein Zufall, sondern ein didaktisch durchdachter Aufbau — und zugleich, wie diese Arbeit zeigt, eine Quelle struktureller Schwierigkeiten.

An Universitäten wie der TU Darmstadt zeigt sich in der pädagogischen Praxis ein problematisches Phänomen, das oft erst bei Änderungen im Lehr- oder Prüfungskontext sichtbar wird. Eigene Beobachtungen im akademischen Umfeld der TU Darmstadt — sowohl als Student als auch im Rahmen der Arbeit — legen nahe, dass Studierende dazu neigen, spezifische Lösungsmuster auswendig zu lernen, anstatt ein tiefgreifendes Verständnis für die zugrundeliegende Hardware-Logik zu entwickeln. Ein konkretes Indiz hierfür liefert die Entwicklung des Notendurchschnitts im Modul „Rechnerorganisation“: In den beobachteten Semestern bis einschließlich WiSe24 lag der Durchschnitt stets bei *3,5* oder besser. Im Sommersemester 2025, das mit einem Professorenwechsel und einer curricularen Umstellung auf die RISC-V-Architektur einherging, sank er auf *4,2*. Wie die Evaluation in Kapitel 7 zeigt, wurde die Qualität der Vorlesung von den Studierenden dabei

ausdrücklich positiv bewertet — der Einbruch ist somit nicht auf eine mangelnde inhaltliche Qualität der Lehre zurückzuführen, sondern deutet auf eine strukturelle Anfälligkeit des erworbenen Wissens gegenüber veränderten Anforderungen hin. Dieses sogenannte „Lernen auf Lücke“ oder Oberflächenlernen führt dazu, dass Wissen bei geringfügigen Abweichungen vom gewohnten Schema nicht mehr transferiert werden kann [28].

Um dieser Problematik entgegenzuwirken, ist es notwendig, die Lernmotivation der Studierenden weg vom rein zielorientierten Auswendiglernen hin zu einem prozessorientierten, tiefen Verständnis zu lenken. Ein vielversprechender Ansatz hierfür ist das Game-Based Learning [27] [4] [35]. Die Gaming-Industrie hat in den letzten Jahrzehnten ein exponentielles Wachstum verzeichnet und eine enorme Reichweite in der jungen Bevölkerungsschicht erreicht, die die primäre Zielgruppe der universitären Lehre darstellt [16] [30]. Statistische Erhebungen lassen die Vermutung zu, dass digitale Spiele heute ein fester Bestandteil der Alltagskultur von Studierenden sind, was ein erhebliches Potenzial für die Wissensvermittlung birgt [50].

In diesem Zusammenhang hat sich das Konzept der Gamifizierung etabliert - die Integration von Spielelementen in spielfremde Kontexte. Während Anwendungen wie Duolingo erfolgreich demonstriert haben, wie Gamifizierung die Nutzerbindung erhöhen kann, zeigen sich bei solchen Ansätzen auch deutliche Grenzen [24]. Kritiker weisen darauf hin, dass bei rein motivierenden Elementen (wie Punkten oder Ranglisten) ein echtes tiefgreifendes Lernen oft ausbleibt und die Nutzer eher auf die Belohnung als auf den Inhalt fokussiert sind. Zudem positionieren sich viele dieser Anwendungen explizit als „Lern-Apps“, was eine bewusste Lernabsicht des Nutzers voraussetzt.

Der Ansatz dieser Arbeit geht jedoch über die klassische Gamifizierung hinaus. Das Ziel ist die Entwicklung eines echten Educational Games. Darunter versteht man eine Software, bei der der Lerninhalt nicht als externe Schicht über die Spielmechanik gelegt wird, sondern zu deren Bestandteil wird: Der Spieler kann das Spiel nicht erfolgreich abschließen, ohne das angestrebte Wissen tatsächlich angewendet zu haben. Der zentrale Unterschied zur Gamifizierung lässt sich an einem konkreten Beispiel verdeutlichen: Eine Plattform, die für das Lösen von Aufgaben zur Prozessorarchitektur Punkte vergibt, betreibt Gamifizierung. Ein Spiel, in dem der Spieler die Prozessorarchitektur selbst verstehen und steuern muss, um voranzukommen, realisiert Game-Based Learning. Im vorliegenden Projekt wird dieser Ansatz konsequent umgesetzt: Die Architektur des Mehrtaktprozessors ist nicht der Hintergrund, sondern der Kern der Spielmechanik — der Spieler agiert als Steuereinheit des Prozessors.

Das Spiel soll ein Eintauchen in einen Flow-Zustand [9] ermöglichen (siehe Kapitel 2.2.2), bei dem der Wissenserwerb nicht als mühsame Pflicht, sondern als notwendiges

Werkzeug zum Meistern spielerischer Herausforderungen wahrgenommen wird. Indem die Architektur des Prozessors selbst zum Kernelement der Spielmechanik wird, soll die intrinsische Motivation der Studierenden geweckt werden, die Funktionsweise der Hardware nicht nur zu reproduzieren, sondern intuitiv zu begreifen.

1.2 Problemstellung

Das Modul „Rechnerorganisation“ an der Technischen Universität Darmstadt stellt für viele Studierende einen signifikanten Paradigmenwechsel in ihrer Ausbildung dar. Erstmals werden sie mit der Assembler-Programmierung konfrontiert - einer Ebene, die sich grundlegend von den zuvor erlernten funktionalen oder objektorientierten Programmierkonzepten unterscheidet. Diese Ebene erfordert nicht nur ein Verständnis für Software, sondern zwingt die Lernenden dazu, kreativ mit den physischen Restriktionen der Hardware umzugehen.

Ein zentrales Problem besteht darin, dass der Erfolg in diesem Modul stark von den Vorkenntnissen aus dem ersten Semester abhängt, insbesondere aus den Modulen „Funktionale und objektorientierte Programmierkonzepte“ sowie „Digitaltechnik“. Wie die Lernforschung zeigt, sind stabile Vorwissensstrukturen eine notwendige Voraussetzung für den erfolgreichen Aufbau komplexen Fachwissens - fehlen sie, entstehen sogenannte Fehlkonzeptionen, die den weiteren Lernprozess systematisch behindern [7]. Wenn die Basisbegriffe der Digitaltechnik nicht sicher beherrscht werden, scheitern die Lernenden in der Rechnerorganisation an der notwendigen Transferleistung: Sie konzentrieren sich so stark auf das Verständnis der Basiselemente, dass die komplexe Kombination dieser Bausteine zu einem Gesamtsystem – dem Mehrtaktprozessor oder Pipelineprozessor – zur unüberwindbaren Hürde wird.

Diese Problematik äußert sich konkret in Aufgabenstellungen, die ein tiefgreifendes Systemverständnis und Eigenständigkeit erfordern. Ein klassisches Beispiel ist die Erweiterung eines bestehenden Datenpfads oder der Finite State Machine der Steuereinheit um einen neuen Befehl. Da viele Studierende dazu neigen, Lösungen für bekannte Aufgabentypen lediglich auswendig zu lernen, fehlt ihnen die Flexibilität, das System auf neue Szenarien zu adaptieren. Statische Lehrmaterialien und Diagramme können hier nur bedingt helfen, da sie kein unmittelbares, formatives Feedback geben. Ein Student merkt oft erst in der Prüfungssituation, dass sein Verständnis der Signalwege fehlerhaft war.

Dieses Phänomen — Studierende kennen die einzelnen Bausteine, vermögen sie aber nicht zu einem kohärenten Gesamtbild zu verknüpfen — entspricht dem in der SOLO-Taxonomie beschriebenen multistrukturalen Verständnis: Lernende beherrschen mehrere Elemente nebeneinander, gelangen jedoch nicht zur relationalen Ebene, auf der diese Elemente in einem sinnvollen Zusammenhang verbunden werden [26].

Zudem führt die visuelle Komplexität des Mehrtaktprozessors zu einer hohen kognitiven Belastung. Aus der Perspektive der Cognitive Load Theory nach Sweller [40], die in Kapitel 2.2.3 ausführlich dargestellt wird, übersteigt die gleichzeitige Verarbeitung von Steuersignalen, Registern und Datenbussen über mehrere Takte hinweg die Kapazität des Arbeitsgedächtnisses erheblich - insbesondere dann, wenn die Lernenden keine Möglichkeit haben, die Informationen durch aktives Handeln zu verankern. Herkömmliche Simulationssoftware bietet hier oft keine Abhilfe, da sie meist eine steile Lernkurve aufweist und eher für die professionelle Entwicklung als für die Didaktik konzipiert ist. Es fehlt an einer Lösung, die den Fokus weg vom passiven „Konsumieren“ von Inhalten hin zu einer interaktiven Erfahrung lenkt. Die Problemstellung dieser Arbeit ergibt sich somit aus der Notwendigkeit, ein Werkzeug zu schaffen, das Wissenslücken schließt, Fehlvorstellungen durch direktes Feedback abbaut und die Studierenden dazu motiviert, sich eigenständig und explorativ mit der Hardware-Architektur auseinanderzusetzen.

1.3 Zielsetzung der Arbeit

Das primäre Ziel der vorliegenden Bachelorarbeit ist die Entwicklung und technische Umsetzung eines mobilen Lernspiels, das auf den Prinzipien des Game-Based Learning basiert. Die Anwendung soll als interaktives Werkzeug dienen, um Studierenden des Moduls „Rechnerorganisation“ sowie verwandter Fachbereiche die komplexen Abläufe innerhalb eines Mehrtaktprozessors zu vermitteln. Im Fokus steht dabei die Transformation theoretischer Lehrinhalte in eine spielerische Umgebung, die ein aktives Experimentieren mit der Hardware-Logik ermöglicht.

Das Kernziel der Arbeit ist es, dass Lernende nach der vollständigen Bearbeitung des Spiels (aktuelle Version - 0.7.6.) eine tiefgreifende Vorstellung vom strukturellen Aufbau sowie vom dynamischen Zusammenspiel der einzelnen Hardware-Komponenten entwickeln. Hierzu gehört insbesondere die Vermittlung fundierter Kenntnisse über die Organisation des Speichers und die unterschiedlichen Befehlstypen der RISC-V-Assembler-Architektur. Ein wesentlicher didaktischer Anspruch des Projekts besteht darin, eine Brücke zwischen der visuellen Simulation im Spiel und dem theoretischen Fundament der Vorlesung zu

schlagen. Dies soll durch eine gezielte Transferleistung erreicht werden, bei der die Nutzer in die Lage versetzt werden, die im Spiel beobachteten Prozesse im Datenpfad direkt auf das Modell der Finite State Machine der Steuereinheit zu mappen. Damit wird das Ziel verfolgt, das oft isolierte Wissen über Zustandsübergänge mit der tatsächlichen physischen Signalverarbeitung zu verknüpfen.

Über die rein didaktischen Ziele hinaus verfolgt die Arbeit eine klare technische Zielsetzung im Bereich der Softwareentwicklung. Die Anwendung wird explizit für das Betriebssystem Android konzipiert und optimiert. Ein integraler Bestandteil der Zielsetzung ist dabei die Vorbereitung des Produkts für eine Veröffentlichung auf gängigen Distributionsplattformen für mobile Anwendungen. Dies umfasst nicht nur die Implementierung einer stabilen Simulations-Logik innerhalb der Unity-Engine, sondern auch die Gestaltung eines Benutzerinterfaces, das trotz der hohen Informationsdichte eines Prozessors eine präzise Steuerung und gute Übersichtlichkeit auf mobilen Endgeräten gewährleistet. Letztlich soll die Arbeit zeigen, dass ein professionell aufbereitetes Lernspiel eine ernsthafte und effektive Ergänzung zur klassischen universitären Lehre darstellt, indem es technische Präzision mit einer motivierenden Nutzererfahrung verbindet.

1.4 Rahmen der Arbeit

Die vorliegende Arbeit bewegt sich in einem spezifisch definierten Rahmen, der sowohl technische als auch inhaltliche Abgrenzungen umfasst. Im Fokus der technischen Realisierung steht die Entwicklung für mobile Endgeräte. Dabei konzentriert sich die aktuelle Umsetzung primär auf das Betriebssystem Android. Diese Entscheidung begründet sich durch die während der Entwicklungsphase begrenzten Hardware-Ressourcen im Bereich der iOS-Endgeräte sowie die restriktiven Richtlinien von Apple bezüglich der App-Moderation und Distribution. Um eine breite Zugänglichkeit für die Evaluation zu gewährleisten, wird das Spiel über den Google Play Store sowie die Plattform itch.io bereitgestellt.

Zusätzlich zur mobilen Version wurde eine funktionsgleiche Fassung für Windows implementiert. Beide Versionen sind hinsichtlich ihres Funktionsumfangs identisch; plattformspezifische Unterschiede in der Implementierung werden detailliert in Kapitel 5.1.4 erläutert. Andere Plattformen, wie Linux, iOS, Web-Browser oder spezialisierte Endgeräte (Konsolen, Smart-TVs), sind zum aktuellen Zeitpunkt nicht Teil des Projektumfangs, stellen jedoch potenzielle Erweiterungsmöglichkeiten für zukünftige Iterationen dar.

Inhaltlich orientiert sich die Applikation eng an den didaktischen Anforderungen der Pflichtmodule „Rechnerorganisation“ und „Digitaltechnik“ im Bachelorstudiengang Informatik an der TU Darmstadt. Als theoretische Grundlage dient hierbei das Standardwerk von Harris & Harris [20]. Das Lernziel ist explizit auf das Thema des Mehrtaktprozessors fokussiert. Flankierende Themengebiete werden nur insoweit behandelt, als sie für das Verständnis des Prozessors zwingend erforderlich sind.

Obwohl die Applikation prinzipiell für eine breite Nutzerschaft zugänglich ist, setzt das didaktische Design einen Kontext voraus, der dem Informatikstudium entspricht. Wie die Evaluation in Kapitel 7 zeigt, können fachfremde Nutzer die App zwar bedienen, benötigen jedoch signifikant mehr Zeit für die Erarbeitung der Themen. Sprachlich ist das Projekt in der aktuellen Version vollständig auf Deutsch (Dialoge, Aufgaben etc.) beschränkt.

1.5 Aufbau der Arbeit

Die vorliegende Arbeit ist strukturiert aufgebaut, um das Projekt von der theoretischen Konzeption bis zur praktischen Evaluation zu beleuchten. Die in Tabelle 1.1 dargestellte Gliederung gibt einen Überblick über die inhaltliche Schwerpunktsetzung der einzelnen Kapitel.

Tabelle 1.1: Inhaltliche Strukturübersicht der Bachelorarbeit nach Kapiteln

Kapitel	**Inhalt**
Einleitung	Führt in die Relevanz des Themas ein, definiert die Problemstellung und die Ziele der Arbeit.
Theoretische Grundlagen	Vermittelt das notwendige theoretische Basiswissen zum Mehrtaktprozessor und den zugrunde liegenden Architekturkonzepten.
Marktanalyse und Abgrenzung	Analysiert bestehende Lehr- und Simulationswerkzeuge und begründet die Positionierung des entwickelten Lernspiels.
Konzeption und Design	Erläutert die didaktischen Ansätze und die gestalterischen Entscheidungen hinter dem Spiel.
Implementierung	Beschreibt die technische Umsetzung in der Unity-Engine sowie die Besonderheiten der Simulations-Logik.
Optimierung	Dokumentiert die grafischen und rechnerischen Optimierungsmaßnahmen für mobile Endgeräte.
Evaluation und Testing	Präsentiert die Ergebnisse der Nutzertests und analysiert den Lernerfolg.
Fazit und Ausblick	Fasst die Ergebnisse zusammen und diskutiert zukünftige Entwicklungsmöglichkeiten.

2 Theoretische Grundlagen

In Kapitel 1.1 wurden die Module „Digitaltechnik“ und „Rechnerorganisation“ bereits kurz eingeführt und ihre Relevanz im Kontext dieser Arbeit skizziert. Im ersten Teil dieses Kapitels wird nun das konkrete Wissen aus beiden Modulen vertieft, das die inhaltliche Grundlage des Lernspiels bildet — mit dem Ziel, dem Leser den nötigen Kontext für das Verständnis der späteren Kapitel zu verschaffen. Im zweiten Teil werden die pädagogischen Konzepte vorgestellt, die das didaktische Design der Anwendung beeinflusst haben.

2.1 Grundlagen der Rechnerarchitektur

2.1.1 Grundlagen der Digitaltechnik

Um einen Mehrtaktprozessor oder einen anderen Prozessor zu verstehen, muss man zunächst verstehen, aus welchen Elementen er aufgebaut ist und nach welchen grundlegenden Gesetzmäßigkeiten diese Elemente funktionieren. Die Komponenten selbst werden in Abschnitt 2.1.3 behandelt; dieser Abschnitt konzentriert sich auf die zugrunde liegenden Prinzipien.

Die Primitive dieser Umgebung sind die logische 0 und die logische 1. Die physikalische Seite dieser Signale - also wie sie entstehen und wie elektrische Spannungspegel als 0 und 1 interpretiert werden — wird in dieser Arbeit ausgespart und kann in Harris & Harris [20], Kapitel 1, nachgelesen werden. Im Rahmen dieser Arbeit werden 0 und 1 als gegebene Konstanten vorausgesetzt, mit denen Operationen durchgeführt werden - insbesondere AND, OR und NOT. Die ersten beiden Operationen werden im Spiel eingeführt und ausführlich erklärt.

Um diese Operationen zu nutzen und aus ihrer Kombination komplexere Strukturen aufzubauen, werden logische Komponenten eingesetzt. Alle logischen Komponenten lassen

sich in zwei Klassen unterteilen: kombinatorische und sequenzielle [20]. Beide Klassen werden im Spiel definiert und ausführlich erläutert. Zusammengefasst gilt: Kombinatorische Komponenten erhalten Eingangssignale und erzeugen daraus ein Ausgangssignal - der Ausgang hängt ausschließlich vom aktuellen Eingang ab. Sequenzielle Komponenten funktionieren ähnlich, besitzen jedoch zusätzlich einen internen Zustand. Das bedeutet, dass das Ausgangssignal nicht nur von den aktuellen Eingaben, sondern auch von den im Bauteil gespeicherten Werten abhängt - was zugleich den zweiten wesentlichen Unterschied markiert: die Fähigkeit, Daten zu speichern.

Doch wann sollen sequenzielle Elemente ihre gespeicherten Daten aktualisieren — und wann nicht? Dafür ist das Taktprinzip verantwortlich. Mit konstanter Periodizität wird an alle sequenziellen Komponenten ein Signal gesendet, das als Takt (Clock) bezeichnet wird. Bei Empfang dieses Signals entscheidet jede sequenzielle Komponente, ob sie ihren gespeicherten Wert aktualisiert, und gibt anschließend ein Signal an die nächste sequenzielle Komponente weiter. Zwischen zwei sequenziellen Komponenten können dabei beliebig viele kombinatorische Komponenten zwischengeschaltet sein, deren einzige Aufgabe es ist, Signale zu verarbeiten und weiterzuleiten. Das Taktprinzip bildet die Grundlage für die Arbeitsweise aller Prozessoren.

Im Spiel erhält der Spieler die Möglichkeit, den Zustand einer Schaltung um beliebig viele Takte zurückzusetzen — bis hin zum Ausgangszustand. Es wird dabei ausdrücklich darauf hingewiesen, dass ein solches „Zurückspulen“ in realen Schaltungen nicht möglich ist: Schaltungen sind darauf ausgelegt, Signale vorwärts weiterzuleiten und ihre definierte Funktion effizient auszuführen — eine Rückverfolgung vorheriger Zustände ist konstruktionsbedingt nicht vorgesehen.

Abschließend ist festzuhalten, dass im Kontext digitaler Systeme Daten, Adressen und Assembler-Instruktionen gleichermaßen als gewöhnliche Zahlen im Speicher abgelegt werden. Eine Interpretation dieser Zahlen findet ausschließlich dann statt, wenn der jeweilige Kontext dies erfordert.

2.1.2 Aufbau des Mehrtaktprozessors

Bevor der strukturelle Aufbau des Mehrtaktprozessors im Detail betrachtet wird, soll zunächst geklärt werden, was ein Mehrtaktprozessor ist, welche Funktion er erfüllt und welche Besonderheiten ihn auszeichnen.

Ein Prozessor ist eine digitale Schaltung, die in der Lage ist, Folgen von Instruktionen — in unserem Fall Assemblerbefehle — aus einem Speicher zu lesen und auszuführen.

Die Architektur eines Prozessors beschreibt, wie seine funktionalen Einheiten organisiert sind und wie Daten zwischen ihnen fließen. Im Rahmen dieser Arbeit, wie auch im Modul „Rechnerorganisation“ an der TU Darmstadt, wird der Mehrtaktprozessor nach Harris & Harris [20] auf der RISC-V-Architektur verwendet.

Der grundlegende Unterschied zwischen einem Eintakt- und einem Mehrtaktprozessor liegt in der Organisation der Befehlsausführung. In einem Eintaktprozessor wird jede Instruktion vollständig innerhalb eines einzigen Taktzyklus abgearbeitet. Der Nachteil dieses Ansatzes besteht darin, dass die Taktperiode auf die langsamste aller möglichen Instruktionen ausgelegt werden muss, was die maximale Taktfrequenz des Gesamtsystems limitiert. Im Mehrtaktprozessor wird die Ausführung einer Instruktion hingegen in mehrere aufeinanderfolgende Schritte (Takte) aufgeteilt, wobei jeder Schritt genau eine definierte Teilaufgabe erfüllt. Dies ermöglicht eine kürzere Taktperiode, da jeder einzelne Schritt weniger Zeit beansprucht als die Gesamtausführung.

Die Ausführung einer Instruktion im Mehrtaktprozessor folgt dabei keinem linearen Schema, sondern einem zustandsabhängigen Ablauf, der nach der Dekodierungsphase je nach Instruktionstyp in unterschiedliche Pfade verzweigt. Der zugehörige Zustandsautomat umfasst insgesamt elf Zustände (S0–S10) und ist in der Abbildung 2.1 dargestellt.

S0 – Fetch: Der Program Counter liefert die Adresse der nächsten Instruktion. Diese wird aus dem Speicher gelesen und im Instruktionsregister abgelegt. Der PC wird für den nächsten Takt aktualisiert. Dieser Zustand wird von jeder Instruktion durchlaufen. S1 – Decode: Die Steuereinheit liest den Opcode der geladenen Instruktion und bereitet die Operanden vor. Der Opcode ist jener Teil des Befehlswortes, der den Instruktionstyp kodiert (siehe Abschnitt 2.1.5). Anhand des Opcodes verzweigt der Automat in einen instruktionsspezifischen Pfad.

Für Lade- und Speicherinstruktionen (lw, sw) folgt zunächst S2 – MemAdr, in dem die Speicheradresse durch die ALU berechnet wird. Anschließend verzweigt der Pfad: Bei lw folgt S3 – MemRead (Lesezugriff auf den Datenspeicher) und danach S4 – MemWB, in dem der gelesene Wert in die Registerbank zurückgeschrieben wird. Bei sw führt der Pfad direkt zu S5 – MemWrite, das den Schreibvorgang abschließt.

Für R-Typ-Instruktionen folgt S6 – ExecuteR, in dem die ALU die geforderte Operation ausführt, gefolgt von S7 – ALUWB, in dem das Ergebnis in die Registerbank geschrieben wird. Denselben Abschlusszustand S7 erreichen auch I-Typ-ALU-Instruktionen über S8 – ExecuteI.

Sprungbefehle bilden eigenständige Pfade: Der unbedingte Sprung jal wird vollständig in S9 – JAL abgehandelt, der bedingte Sprung beq in S10 – BEQ. Nach Abschluss jedes

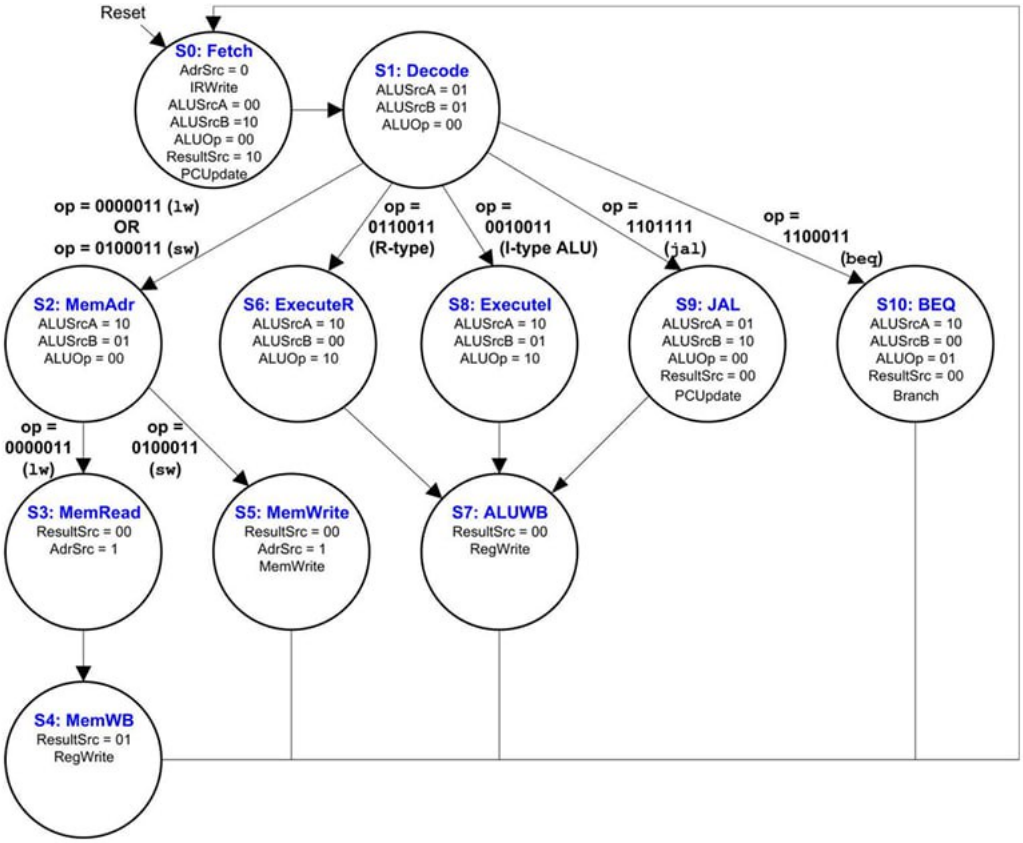


Abbildung 2.1: Finite state Machine von Mehrtaktprozessor. Quelle - Harris & Harris [20]

Pfades kehrt der Automat in den Ausgangszustand S0 zurück und beginnt den Zyklus mit der nächsten Instruktion.

2.1.3 Komponenten des Datenpfads

Der Datenpfad bezeichnet die Gesamtheit aller funktionalen Einheiten und Verbindungen, über die Daten im Prozessor transportiert und verarbeitet werden. Die nachfolgende Beschreibung bezieht sich auf die konkrete Implementierung nach Harris & Harris [20], wie sie auch der Spielsimulation zugrunde liegt.

Program Counter (PC): Ein getaktetes Register, das die Adresse der aktuell auszuführenden Instruktion enthält. Das Enable-Signal PCWrite steuert, ob der PC am Ende eines Taktes seinen Wert aktualisiert — notwendig, da der PC nicht in jedem Takt überschrieben werden darf, etwa während eines Speicherzugriffs. Ein zusätzliches Register OldPC speichert den vorherigen PC-Wert, der für die Berechnung von Sprungadressen benötigt wird.

Kombinierter Instruktions- und Datenspeicher: Diese Architektur verwendet einen einzigen gemeinsamen Speicher für Instruktionen und Daten. Das Steuersignal AdrSrc bestimmt, welche Adresse an den Speicher weitergeleitet wird: bei AdrSrc = 0 die Adresse des PC (Instruktionsabruf), bei AdrSrc = 1 das berechnete ALU-Ergebnis (Datenzugriff). Das Signal IRWrite steuert, ob der gelesene Wert im Instruktionsregister abgelegt wird — dies geschieht ausschließlich im Fetch-Zustand.

Registerbank (Register File): Die Registerbank enthält 32 Allzweckregister (x0–x31) mit einer Breite von 32 Bit. Sie verfügt über zwei Leseports (RD1, RD2), adressiert über Rs1 (Bits 19:15) und Rs2 (Bits 24:20), sowie einen Schreibport (WD3), gesteuert über Rd (Bits 11:7) und das Signal RegWrite. Das Register x0 ist fest auf den Wert 0 verdrahtet.

Arithmetisch-Logische Einheit (ALU): Die ALU empfängt zwei Operanden (SrcA, SrcB) und führt die durch ALUControl kodierte Operation aus. SrcA wird über einen Multiplexer aus drei Quellen gewählt: dem aktuellen PC-Wert (ALUSrcA = 00), dem OldPC-Wert (ALUSrcA = 01) oder dem Registerausgang RD1 (ALUSrcA = 10). SrcB wird analog aus RD2, dem Immediate-Wert oder dem Konstantenwert 4 gewählt. Das Ergebnis steht sowohl direkt als ALUResult als auch zwischengespeichert als ALUOut zur Verfügung.

Sign-Extend-Einheit (Extend): Extrahiert den Immediate-Wert aus den Bits 31:7 der Instruktion und erweitert ihn vorzeichengerecht auf 32 Bit. Das Signal ImmSrc bestimmt das Kodierungsformat, da verschiedene Instruktionstypen die Konstante in unterschiedlichen Bitfeldern ablegen.

Ergebnis-Multiplexer (ResultSrc): Wählt das Ergebnis für den Registerbank-Schreibport: ResultSrc = 00 liefert ALUResult, ResultSrc = 01 den Speicherlesewert (ReadData), ResultSrc = 10 den OldPC-Wert für Sprungbefehle.

Steuereinheit (Control Unit): Empfängt als Eingaben den Opcode (Bits 6:0), funct3 (Bits 14:12), funct7 (Bit 30) sowie das Zero-Flag der ALU und erzeugt daraus alle Steuersignale des Datenpfads. Ihre interne Realisierung als Finite State Machine wird in Abschnitt 2.1.4 beschrieben.

2.1.4 Die Steuereinheit und das Prinzip der Finite State Machine

Die Steuereinheit (Control Unit) ist die Komponente des Prozessors, die den Datenpfad koordiniert. Sie empfängt den Opcode der aktuellen Instruktion sowie den aktuellen Ausführungszustand und erzeugt daraus die Steuersignale für alle Multiplexer, Speicher und Register. Da die Steuereinheit im Mehrtaktprozessor den aktuellen Taktschritt berücksichtigen muss, ist sie als sequenzielle Schaltung realisiert.

Das formale Modell für solche zustandsabhängigen Systeme ist die Finite State Machine (FSM), auf Deutsch endlicher Automat. Eine FSM besteht aus einer endlichen Menge von Zuständen, einer Menge von Eingaben, einer Übergangsfunktion, die den nächsten Zustand in Abhängigkeit von aktuellem Zustand und Eingabe bestimmt, sowie einer Ausgabefunktion, die die Steuersignale je nach aktuellem Zustand erzeugt. Im Mehrtaktprozessor entspricht jeder Zustand einem konkreten Ausführungsschritt — dem Fetch-Zustand, dem Decode-Zustand oder einem instruktionsspezifischen Execute-Zustand. Der Übergang zwischen Zuständen erfolgt mit jeder steigenden Taktflanke.

Sowohl nach persönlicher Erfahrung im Rahmen dieser Arbeit als auch nach Beobachtungen der Fachgruppe ESA an der TU Darmstadt stellt das Verständnis dieser Zustandsmaschine für Studierende eine besondere Herausforderung dar: Es erfordert die Fähigkeit, abstrakte Zustandsübergänge mit konkreten physischen Signalzuständen im Datenpfad zu verknüpfen. Genau diese Transferleistung bildet das didaktische Kernziel des entwickelten Lernspiels, dessen konkrete Umsetzung in Kapitel 4.2 beschrieben wird.

2.1.5 RISC-V Instruktionsarchitektur

RISC-V ist eine offene Befehlssatzarchitektur (Instruction Set Architecture, ISA), die auf dem Prinzip des Reduced Instruction Set Computing (RISC) basiert. Im Gegensatz zu

proprietären Architekturen wie x86 oder ARM ist RISC-V lizenzfrei und wird zunehmend in der akademischen Lehre eingesetzt — unter anderem an der TU Darmstadt im Modul „Rechnerorganisation“.

RISC-V-Instruktionen sind einheitlich 32 Bit breit. Für den Mehrtaktprozessor im Rahmen dieser Arbeit sind folgende Instruktionstypen relevant:

R-Typ (Register-Register): Arithmetische und logische Operationen zwischen zwei Quellregistern, deren Ergebnis in ein Zielregister geschrieben wird. Beispiele: add, sub, and, or.

I-Typ (Immediate): Operationen zwischen einem unmittelbaren Konstantenwert und einem Registerwert, deren Ergebnis in ein Zielregister geschrieben wird, sowie Ladeoperationen aus dem Speicher. Beispiele: addi, lw.

S-Typ (Store): Speicheroperationen, die den Inhalt eines Registers in den Datenspeicher schreiben. Beispiel: sw.

B-Typ (Branch): Bedingte Sprungbefehle, die den Program Counter auf eine neue Adresse setzen, wenn eine bestimmte Bedingung erfüllt ist. Beispiel: beq.

J-Typ (Jump): Unbedingter Sprung zu einer Zieladresse. Beispiel: jal.

Jedes Format kodiert den Opcode, die Registeradressen und gegebenenfalls den Immediate-Wert in fest definierten Bitfeldern des 32-Bit-Befehlswortes. Die Steuereinheit interpretiert den Opcode und konfiguriert den Datenpfad entsprechend dem jeweiligen Instruktionstyp.

2.2 Pädagogische Grundlagen

2.2.1 Gamification und Game-Based Learning

Im Diskurs über den Einsatz spielerischer Elemente in Bildungskontexten ist eine begriffliche Abgrenzung zwischen Gamification und Game-Based Learning (GBL) unerlässlich, da beide Konzepte häufig synonym verwendet werden, obwohl sie grundlegend unterschiedliche Ansätze verfolgen.

Gamification bezeichnet die Integration spieltypischer Elemente - wie Punkte, Abzeichen, Ranglisten oder Fortschrittsbalken - in einen nicht-spielerischen Kontext [11]. Das Ziel ist primär die Steigerung der Motivation und Nutzerbindung durch extrinsische Anreize. Erfolgreiche Beispiele wie Duolingo zeigen, dass dieser Ansatz die Nutzungsfrequenz

erhöhen kann. Forschungsergebnisse deuten jedoch darauf hin, dass Gamification zwar die Motivation steigert, ein tiefgreifendes konzeptuelles Verständnis jedoch nur bedingt fördert — insbesondere dann, wenn die Belohnung von der eigentlichen Lernaufgabe entkoppelt ist [24].

Game-Based Learning geht einen grundlegend anderen Weg: Hier ist der Lerninhalt selbst die Spielmechanik. Der Wissenserwerb ist nicht ein Mittel zur Erlangung einer Belohnung, sondern eine notwendige Voraussetzung für das Meistern der Spielherausforderungen [4]. Der Unterschied lässt sich an einem Beispiel verdeutlichen: Eine Lernplattform, die für das Lösen von Mathematikaufgaben Punkte vergibt, betreibt Gamification. Ein Spiel, in dem der Spieler mathematische Prinzipien anwenden muss, um Rätsel zu lösen und voranzukommen, realisiert Game-Based Learning. Im vorliegenden Projekt wird dieser Ansatz konsequent umgesetzt: Die Architektur des Mehrtaktprozessors ist nicht der Hintergrund, sondern der Kern der Spielmechanik — der Spieler agiert als Steuereinheit des Prozessors.

2.2.2 Flow-Theorie

Das Konzept des Flow-Zustands wurde vom Psychologen Mihaly Csikszentmihalyi beschrieben als ein Zustand vollständiger mentaler Versunkenheit in eine Tätigkeit, der von hoher intrinsischer Motivation und dem Verlust des Zeitgefühls begleitet wird [9] [19]. Flow entsteht dann, wenn der Schwierigkeitsgrad einer Aufgabe und die Fähigkeiten des Ausführenden in einem optimalen Gleichgewicht stehen: Ist die Aufgabe zu einfach, entsteht Langeweile; ist sie zu schwierig, entsteht Frustration oder Angst. Nur im Bereich zwischen diesen Extremen — dem sogenannten Flow-Kanal — ist nachhaltiges, motiviertes Engagement möglich.

Für das Design von Lernspielen hat dieses Modell direkte Implikationen. Ein Lernspiel muss den Schwierigkeitsgrad dynamisch an den Lernfortschritt des Nutzers anpassen, um den Spieler dauerhaft im Flow-Kanal zu halten. Im vorliegenden Projekt wird dies durch einen inkrementellen Lernpfad realisiert, der mit einfachen, vertrauten Komponenten beginnt und die Komplexität schrittweise steigert. Darüber hinaus unterstützt die technische Stabilität der Anwendung den Flow-Zustand: Ruckler, Ladezeiten oder Verzögerungen in der Benutzeroberfläche unterbrechen die kognitive Versunkenheit und sind daher aus didaktischer Perspektive nicht tolerierbar — ein Umstand, der die Optimierungsmaßnahmen in Kapitel 6 motiviert.

2.2.3 Cognitive Load Theory nach Sweller

Die Cognitive Load Theory (CLT) wurde von John Sweller entwickelt und beschreibt die Grenzen des menschlichen Arbeitsgedächtnisses bei der Verarbeitung neuer Informationen [40]. Die Theorie unterscheidet drei Arten kognitiver Belastung: Intrinsic Load bezeichnet die inhärente Komplexität des Lernmaterials selbst — sie ist abhängig vom Vorwissen des Lernenden und der Anzahl der Elemente, die gleichzeitig verarbeitet werden müssen. Die Architektur des Mehrtaktprozessors besitzt einen hohen intrinsic load, da sie das simultane Verfolgen mehrerer Signalpfade, Zustände und Komponenten erfordert.

Extraneous Load bezeichnet die zusätzliche kognitive Belastung, die durch die Art der Informationsdarbietung — also durch schlechtes didaktisches Design — entsteht. Unübersichtliche Benutzeroberflächen, inkonsistente Farbkodierungen oder überladene Diagramme erhöhen den extraneous load, ohne zum Lernziel beizutragen. Germane Load bezeichnet die kognitive Aktivität, die unmittelbar dem Aufbau von Wissensstrukturen (Schemata) dient. Gut gestaltetes Lernmaterial maximiert den germane load bei gleichzeitiger Minimierung des extraneous load.

Für das Design des Lernspiels ergibt sich daraus die Forderung, die visuelle und interaktive Gestaltung so zu optimieren, dass die kognitive Ressourcen des Lernenden primär für das Verstehen der Prozessorlogik aufgewendet werden. Konkret wird dies durch die Aufteilung der Lerninhalte auf einzelne, fokussierte Level realisiert, in denen jeweils nur eine neue Komponente eingeführt wird, sowie durch ein konsistentes Farbkodierungssystem, das Signalzustände intuitiv kommuniziert.

2.2.4 Scaffolding als didaktisches Prinzip

Der Begriff Scaffolding — „Lehrgerüst“ auf Deutsch — bezeichnet eine didaktische Strategie, bei der Lernende beim Erwerb neuer Fähigkeiten durch gezielte, temporäre Unterstützungsmaßnahmen begleitet werden, die schrittweise reduziert werden, sobald das Verständnis gefestigt ist [52]. Das Konzept geht auf die Arbeiten von Lew Vygotsky zurück, insbesondere auf dessen Konzept der Zone der nächsten Entwicklung (Zone of Proximal Development): den Bereich zwischen dem, was ein Lernender selbstständig bewältigen kann, und dem, was er mit Unterstützung erreichen kann [49].

Im Kontext des Lernspiels manifestiert sich Scaffolding auf mehreren Ebenen. Auf inhaltlicher Ebene beginnt das Spiel mit Komponenten, die aus dem Modul „Digitaltechnik“ bereits bekannt sind — Registern, Multiplexern und der ALU — und führt diese als eigenständige

Lerneinheiten ein, bevor sie als Bestandteile des Mehrtaktprozessors zusammengesetzt werden. Auf interaktiver Ebene stehen dem Spieler in frühen Leveln kontextabhängige Hinweise zur Verfügung, die bei Bedarf aktiviert werden können und die kognitive Last beim erstmaligen Umgang mit neuen Konzepten reduzieren. Mit zunehmendem Spielfortschritt werden diese Unterstützungsmaßnahmen schrittweise zurückgezogen, bis der Spieler im finalen Kapitel die vollständige Prozessorsimulation selbstständig bedient.

3 Marktanalyse und Abgrenzung

Zur Einordnung des entwickelten Lernspiels in das bestehende Angebot wird in diesem Kapitel eine Analyse relevanter Softwarelösungen vorgenommen. Die betrachteten Lösungen lassen sich in drei Kategorien unterteilen: spielerische Simulationen für Desktop-Plattformen, akademische Simulationswerkzeuge sowie Anwendungen im mobilen Segment.

3.1 Spielerische Hardware-Simulationen (Desktop)

Im Bereich der spielerischen Hardware-Simulationen hat sich auf der Distributionsplattform Steam eine Reihe von Titeln etabliert, die das Thema Prozessorarchitektur oder digitale Logik in einem spielerischen Format aufgreifen. Die nachfolgende Tabelle 3.1 gibt einen Überblick über die relevantesten Vertreter dieses Segments, bewertet anhand von Preis, Nutzerbasis sowie inhaltlicher Ausrichtung:

Tabelle 3.1: Vergleich relevanter Spiele auf der Plattform Steam

Titel	Preis	Anzahl Rez.	RISC-V Basis	Mehrtakt- prozessor	Assembler-bezug
Turing Complete [23]	16,79 €	∼5.000	Nein	Nur einfache CPU	Ja
Shenzhen I/O [53]	14,79 €	∼3.700	Nein	Nein	Ja
MHRD [15]	9,99 €	∼423	Nein	Nur einfache CPU	Nein
Silicon Zeroes [17]	14,79 €	∼247	Nein	Nur einfache CPU	Ja

Die Analyse dieser Projekte deutet auf zwei Einschränkungen. Erstens sind diese Projekte kostenpflichtig - keiner unterschreitet die Marke von 9,99 €, was für Studierende eine relevante Zugangshürde darstellt. Zweitens definieren diese Spiele entweder eine eigene, fiktive Architektur oder beschränken sich auf grundlegende CPU-Konzepte ohne curricularen Bezug zur RISC-V-Architektur. So verwendet etwa Shenzhen I/O [53] eine proprietäre Assemblersprache, die zwar allgemeines Verständnis fördert, bei Studierenden der TU Darmstadt jedoch zu Verwirrung führen kann, da das erworbene Wissen nicht direkt auf die Lehrinhalte des Moduls „Rechnerorganisation“ übertragbar ist. Keiner der analysierten Titel behandelt explizit die Architektur des Mehrtaktprozessors.

Darüber hinaus sind diese Spiele als offene Sandboxen konzipiert: Der Spieler wird mit den Grundbausteinen konfrontiert und muss sich das System eigenständig erarbeiten, ohne strukturierte didaktische Begleitung. Ein Spieler, der gezielt die Architektur des Mehrtaktprozessors nach Harris & Harris [20] verstehen möchte, findet keinen direkten Einstiegspunkt und muss nach dem Durchlaufen des gesamten Spiels weiterhin auf Vorlesungsfolien zurückgreifen.

Im Gegensatz dazu ist das im Rahmen dieser Arbeit entwickelte Projekt auf allen Plattformen kostenlos verfügbar, auf die Anforderungen der TU Darmstadt ausgerichtet und begleitet den Lernenden durch einen strukturierten Lernpfad mit theoretischen Einführungen und kontextuellen Hinweisen auf jedem Level.

3.2 Akademische Simulationswerkzeuge (Desktop)

Neben spielerischen Titeln existieren im akademischen Umfeld frei verfügbare Simulationswerkzeuge, die im Rahmen der Lehrveranstaltungen eingesetzt oder empfohlen werden. Ein bemerkenswerter Vertreter ist Ripes [34] — ein visueller Simulator, der auf der RISC-V-Architektur basiert und eine detaillierte Analyse von Datenpfaden sowie Pipeline-Stufen ermöglicht. Auf den ersten Blick erscheint Ripes als direkter Konkurrent: Es ist kostenlos, RISC-V-basiert und bietet tiefgehende analytische Werkzeuge.

Dennoch ist anzumerken — basierend auf persönlichen Beobachtungen im akademischen Umfeld der TU Darmstadt sowie Gesprächen mit Verantwortlichen der Fachgruppe ESA —, dass Studierende solche Werkzeuge trotz expliziter Empfehlungen im Rahmen des Kurses kaum nutzen. Als wahrscheinlichste Ursache wurde die Installationshürde identifiziert: Studierende greifen bevorzugt auf webbasierte Lösungen zurück, da diese ohne Einrichtungsaufwand zugänglich sind. Es liegt nahe, dass jegliche Einstiegshürde — selbst

eine kurze Installationsroutine — die Nutzungsbereitschaft signifikant senkt, da sie eine zusätzliche kognitive und zeitliche Belastung darstellt.

Ein ähnliches Phänomen zeigt sich bei Nand2Tetris [38]: Das Projekt führt Lernende von elementaren Logikgattern bis hin zur Implementierung eines vollständigen Computers. Trotz seines didaktischen Anspruchs setzt es eine eigene Hardwarebeschreibungssprache voraus und ist nicht auf die spezifischen Lerninhalte der TU Darmstadt ausgerichtet. Das vorliegende Lernspiel hingegen ist über etablierte Distributionsplattformen mit einem einzelnen Klick installierbar, in logische Kapitel mit begleitenden Erklärungen gegliedert und lässt den Lernenden zu keinem Zeitpunkt ohne Orientierung. Die einzige kognitive Belastung, die dem Spieler zugemutet wird, ist das Verständnis des Lehrinhalts selbst.

3.3 Mobiles Segment

Im mobilen Segment sind kaum Anwendungen zu finden, die dem Themenbereich dieser Arbeit entsprechen. Zwei Projekte sind dennoch erwähnenswert. Human Resource Machine [8] ist ein qualitativ hochwertiger Port einer erfolgreichen Desktop-Anwendung, der das Thema Assemblerprogrammierung in einer eigenen fiktiven Architektur behandelt. Die Anwendung ist jedoch kostenpflichtig und deckt die Architektur des Mehrtaktprozessors nicht ab. Circuit Snap [36] konzentriert sich auf grundlegende Logikkomponenten, ist nur eingeschränkt kostenlos verfügbar und hat keine nennenswerte Nutzerbasis erreicht.

In beiden Fällen grenzt sich das vorliegende Projekt durch seine kostenlose Verfügbarkeit, seine curriculare Spezifität und seinen strukturierten Lernpfad klar ab.

3.4 Abgrenzung des Projekts

Obwohl das Projekt auf dem analysierten Markt eine klare Lücke schließt, sind die Grenzen des aktuellen Entwicklungsstands transparent zu benennen.

Technische Grenzen
Zum Zeitpunkt der Einreichung dieser Arbeit ist die Anwendung für Android und Windows verfügbar. Eine Erweiterung auf iOS, Linux und Web ist für zukünftige Iterationen vorgesehen. Spielkonsolen und spezialisierte Endgeräte sind nicht Teil des geplanten Umfangs.

Didaktische Grenzen
Die Anwendung vermittelt gezielt die Architektur des Mehrtaktprozessors im Kontext des Moduls „Rechnerorganisation“. Verwandte Themen wie Eintaktprozessor, Pipelineprozessor oder digitale Schaltungen werden nur insoweit behandelt, als sie für das Verständnis des Mehrtaktprozessors notwendig sind. Das volle didaktische Potenzial entfaltet das Spiel bei Studierenden der TU Darmstadt, die das entsprechende Modul (Digitaltechnik) belegen oder bereits absolviert haben, da das Design einen universitären Vorwissenskontext voraussetzt. Fachfremde Nutzende können die Anwendung zwar durchspielen, benötigen dafür jedoch mehr Zeit und kognitive Ressourcen — ein Befund, der durch die Nutzertests in Kapitel 7.2.2 bestätigt wurde.

4 Konzeption und Design

4.1 Game Design Document

Die konzeptionelle Grundlage des Projekts bildet ein Game Design Document (GDD), das als Plan für die gesamte Entwicklung diente. Das primäre Spielziel besteht im sequenziellen Abschluss von Leveln durch das Lösen technischer Aufgaben an verschiedenen Schaltungen. Während das unmittelbare Spielziel auf das Abschließen der Level ausgerichtet ist, verfolgt das strategische Bildungsziel des Projekts eine tiefergehende Absicht: das Erlernen beziehungsweise Festigen der Kenntnisse über sämtliche Komponenten des Mehrtaktprozessors und deren Zusammenspiel als kohärentes Gesamtsystem.

Der zentrale Spielkreislauf (Core Loop) wurde so konzipiert, dass er eine kontinuierliche Spielerbindung gewährleistet und gleichzeitig eine effektive Wissensvermittlung ermöglicht [10]. Er gliedert sich in vier zyklisch wiederkehrende Phasen. Zunächst erfolgt die Aktualisierung des Vorwissens durch die Interaktion mit dem Spielcharakter „Charlie“, der den Spieler in dialogischer Form in den Kontext des Levels einführt und das benötigte theoretische Fundament erläutert oder in Erinnerung ruft [51]. Es folgt ein analytischer Schritt, in dem der Spieler die vorliegende Schaltung des Levels untersucht und die gestellte Aufgabe dekonstruiert - er muss die erhaltene Theorie auf das visuelle Datenmodell abbilden. In der aktiven Lösungsphase manipuliert der Spieler die Steuersignale, um den geforderten Schaltzustand zu erreichen; kontextabhängige Hinweise reduzieren dabei die Einstiegshürde bei komplexen Knoten. Den Abschluss bildet die Progression: die erfolgreiche Validierung der Lösung und der Übergang zur nächsten, komplexeren Lerneinheit.

Die Zielgruppe des Projekts ist klar definiert: Es handelt sich um Bachelorstudierende der Technischen Universität Darmstadt aus den Fachbereichen Informatik und Elektrotechnik und Informationstechnik, insbesondere Teilnehmende der Lehrveranstaltungen „Rechnerorganisation“ und verwandter Module. Diese enge Ausrichtung ermöglichte eine

präzise Anpassung der Aufgabenkomplexität und der verwendeten Terminologie an das akademische Niveau der Universität [5].

Aus spieltypologischer Sicht stellt das Projekt einen Hybrid aus Educational Game, Puzzle und Simulation dar. Die Simulationskomponente gewährleistet die präzise Nachbildung des Verhaltens logischer Schaltungen. Das Puzzle-Format stimuliert kritisches Denken und die eigenständige Lösungsfindung. Die übergeordnete Bildungsorientierung transformiert die Spielerfahrung in ein effektives Vorbereitungswerkzeug für Prüfungen und praktische Übungen [42].

4.2 Didaktisches Level-Design

Scaffolding und Struktur des Lernpfads
Die zentrale Herausforderung bei der Entwicklung der Level bestand in der Dekonstruktion der Mehrtaktprozessor-Architektur - eines hochkomplexen Systems, das das Zusammenführen von Kenntnissen aus verschiedenen Teilgebieten der Informatik erfordert. Um eine effektive Wissensaneignung zu ermöglichen, stützt sich das Projekt auf die Methode des pädagogischen Scaffolding [25]. Dieser Ansatz sieht vor, neue Konzepte schrittweise auf Basis bereits gefestigten Wissens einzuführen.

Da der Mehrtaktprozessor tiefgreifende Vorkenntnisse voraussetzt, wurde bewusst entschieden, den gesamten benötigten Wissensbedarf innerhalb des Spiels abzudecken. Das bedeutet, dass selbst grundlegende Komponenten aus dem Modul „Digitaltechnik“ vollständig wiederholt werden. Dieser Ansatz erwies sich als wirkungsvoll: Mindestens zwei Testpersonen gaben an, dass das Spiel ihnen half, Fehlvorstellungen zu korrigieren, die sie im Rahmen des Moduls „Digitaltechnik“ entwickelt hatten [37].

Das Spiel ist in sieben Kapitel gegliedert: Register, Multiplexer, ALU, Speicher, Extender, Register File und Mehrtaktprozessor. Die Reihenfolge der Kapitel folgt einer didaktischen Logik. Das erste Kapitel widmet sich Registern, da diese von Studierenden häufig missverstanden werden. Um eine möglichst flache Lernkurve zu gewährleisten, werden Register bereits zu Beginn eingeführt und begleiten den Spieler durch alle weiteren Kapitel, was ihr Verständnis durch Wiederholung festigt. Da Multiplexer und ALU erfahrungsgemäß weniger Verständnisprobleme bereiten, sind diese Kapitel bewusst kürzer und leichter gehalten — sie dienen der Aktivierung bereits vorhandenen Wissens, ohne den Lernenden aufzuhalten. Der letzte Level des ALU-Kapitels führt ein neues, fundamentales Konzept

ein: die Grundlage zur Aktualisierung des Program Counters sowie das Prinzip der geschlossenen Schleife, bei der der Ausgang einer Schaltung gleichzeitig ihr Eingang ist. Dieses Prinzip wird in allen nachfolgenden Kapiteln aktiv verwendet und muss daher frühzeitig verankert werden.

Steuerung der kognitiven Belastung
Grundlage der Levelplanung ist die Cognitive Load Theory nach Sweller [40], die eine bewusste Steuerung der Informationsmenge je Lerneinheit fordert [13]. Komplexe Komponenten wie der Speicher werden daher isoliert erarbeitet: Jeder Eingang und Ausgang wird auf einem separaten Level behandelt, sodass der Spieler nicht den vollen Funktionsumfang eines Moduls gleichzeitig im Arbeitsspeicher halten muss [29].

Da das Kapitel über den Speicher bereits inhaltlich dicht ist, werden die ersten Assembler-Instruktionen erst im Extender-Kapitel eingeführt. Zum Abschluss dieses Kapitels werden die Funktionsweise des Speichers, die Interaktion mit dem Extender und der Assembler-Kontext zusammengeführt. Ein ähnliches Prinzip findet im Register-File-Kapitel Anwendung: Jeder Level führt neue Eingänge ein und klärt deren Zuständigkeit in Abgrenzung zu den übrigen Signalen. In den letzten Leveln dieses Kapitels werden Assembler und Extender integriert. In den Kapiteln Register File und Extender erlernt der Spieler zudem die Kamerasteuerung, die für die Inspektion des vollständigen Mehrtaktprozessors im Abschlusskapitel notwendig ist.

Im Abschlusskapitel wechselt das Spiel in einen Sandbox-Modus, der es dem Studierenden ermöglicht, alle erlernten Instruktionstypen aktiv zu erproben. Um die Aufmerksamkeit auf die Verarbeitungslogik zu lenken, ist der Program Counter auf den meisten Leveln bereits korrekt voreingestellt. Ein spezieller Level setzt bewusst einen falschen Anfangswert im PC, um den Spieler mit der manuellen Aktualisierung des Program Counters vertraut zu machen.

Ein weiteres Gestaltungsmerkmal betrifft die Transparenz des Leveldesigns: Den Spielern wird nicht kommuniziert, welche Level Teilsegmente des vollständigen Prozessors darstellen. Diese Entscheidung wurde bewusst getroffen, um zu verhindern, dass der Spieler die vermeintliche Komplexität eines Levels antizipiert und sich dadurch blockiert. Stattdessen werden die Kernblöcke des Prozessors beiläufig und sukzessiv erschlossen.

Fehlermechanik und formatives Feedback
Die Spielmechanik bietet dem Lernenden vollständige Transparenz über den aktuellen Zustand der Schaltung [14]. Sämtliche Signalwerte sind jederzeit einsehbar; während der Animationsphasen zwischen den Takten werden die auf den Datenbussen fließenden Werte dynamisch visualisiert. Der Spieler kann zu jedem Zeitpunkt die Korrektheit seiner Lösung

überprüfen und erhält unmittelbar Rückmeldung [21]. Stellt er einen Fehler fest oder möchte eine Anpassung vornehmen, kann er einen Takt zurückspringen. Da jeder Level bei Takt null beginnt, ist ein vollständiger Reset jederzeit möglich. Dieser Mechanismus des iterativen Fehlerkorrigierens ermöglicht es dem Nutzer, Ursache-Wirkungs-Zusammenhänge in der Prozessorlogik selbstständig zu erschließen und macht den Debugging-Prozess zum Lerninstrument.

4.3 Visueller Stil und Ästhetik

Ein verbreitetes Problem bei Lernspielen ist ihre übermäßige Funktionalitätsorientierung auf Kosten der visuellen Attraktivität, was dazu führt, dass sie von Nutzern als „Lernprogramm“ statt als vollwertiges Spiel wahrgenommen werden. Das Ziel dieses Projekts war es, das bewusste Lerngefühl zu minimieren und den Spieler in einen Flow-Zustand zu versetzen [9] [19]. Zu diesem Zweck wurde ein visueller Stil entwickelt, der Fantasy-Elemente mit einer natürlichen, organischen Ästhetik verbindet. Diese Themenwahl ermöglichte eine bewusste Distanzierung von den sterilen, technischen Assoziationen, die Informatik typischerweise hervorruft, und schafft gleichzeitig Raum für narrative Weiterentwicklung.

Trotz der angestrebten atmosphärischen Dichte liegt dem Design das Prinzip des funktionalen Minimalismus zugrunde. Da die Architektur des Mehrtaktprozessors selbst bereits informationsdicht und kognitiv fordernd ist, würde übermäßige visuelle Detaillierung unweigerlich zu einem Cognitive Overload führen. Der Minimalismus reduziert das visuelle Rauschen und lenkt die Aufmerksamkeit des Spielers auf die Logik der Komponenten, wie in Abbildung 4.1 anhand eines „Extender-Levels“ zu sehen ist. Darüber hinaus vereinfachte die Verwendung einfacher geometrischer Formen den Produktionsprozess der 3D-Modelle erheblich.

Ein kritischer Aspekt des Designs ist das Farbkodierungssystem. Um dekoratives Umfeld und funktionale Spielobjekte intuitiv zu trennen, wurde eine strikte Farbpalette eingeführt. Alle interaktiven Prozessorkomponenten — Register, ALU und verwandte Elemente — sind in Weiß gehalten. Wie die Abbildung 4.2 verdeutlicht, bilden sie damit das einzige weiße Element der Szene, was dem Spieler eine sofortige visuelle Identifikation interagierbarer Objekte ermöglicht.

Inaktive Datenpfade und Konstanten werden in Dunkelblau dargestellt; aktive Signalwege leuchten in einem hellen Blau auf. Dieses System erlaubt es, den Zustand von Multiplexern



Abbildung 4.1: Darstellung eines Extender-Levels. Ausgewogene visuelle Gestaltung, die trotz Detailreduktion eine immersive Atmosphäre vermittelt.

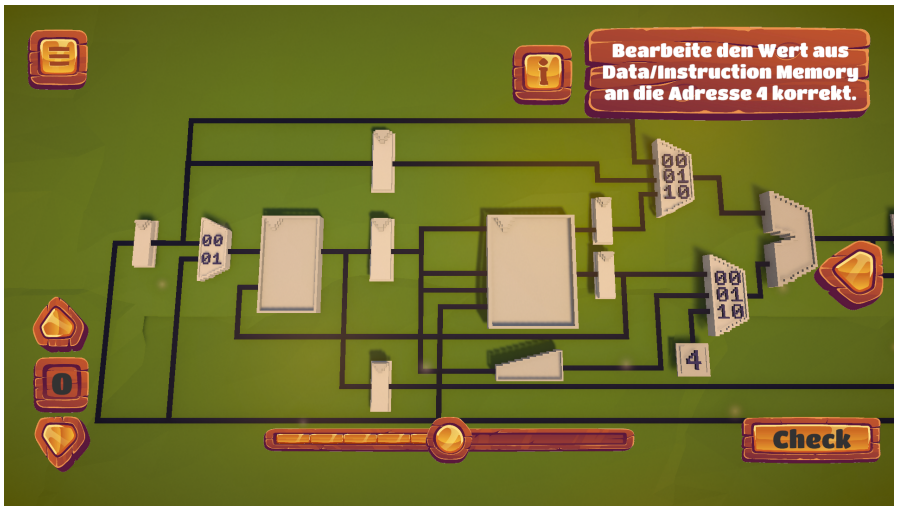


Abbildung 4.2: Akzentuierung interaktiver Komponenten durch hohen Helligkeitskontrast gegenüber der Hintergrundumgebung.

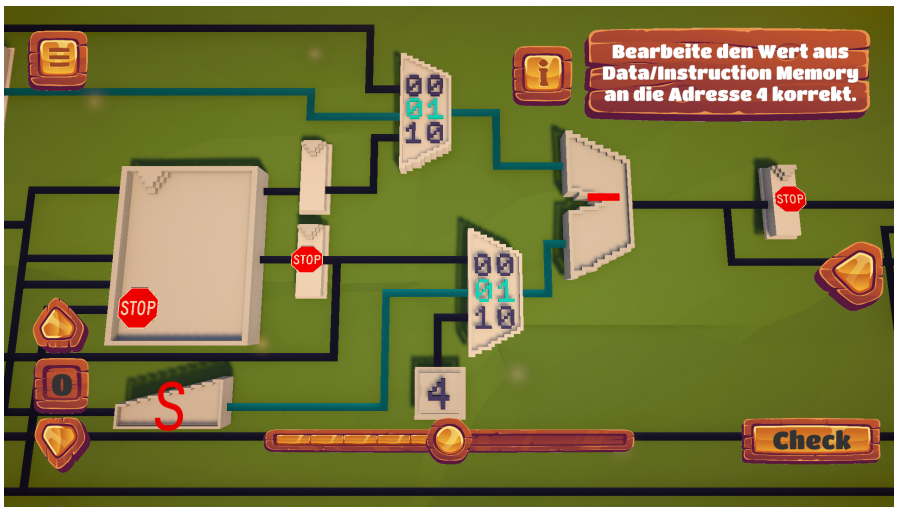


Abbildung 4.3: Auf diesem Bild ist zu sehen, wie Multiplexer, Extender und ALU im Spiel eingesetzt werden. Außerdem ist zu erkennen, dass für einige Register das Schreiben neuer Werte gesperrt ist.

und den Signalfluss zu verfolgen, ohne das Arbeitsgedächtnis zu belasten (siehe Abbildung 4.3).

Im Entwicklungsprozess durchlief die Adaptation klassischer Schaltungsdiagramme — in Anlehnung an *Harris & Harris* [20] — eine iterative Phase. Ein früher Entwurf, bei dem Prozessorkomponenten durch architektonische Objekte ersetzt wurden (etwa ein Rathaus als ALU), wurde im Rahmen von Nutzertests verworfen: Die Detailliertheit der Gebäude führte zu Verwirrung über die Funktion der Objekte. Es wurde schließlich entschieden, die aus den Lehrbüchern bekannten Umrisse der Komponenten beizubehalten. Dies gewährleistet ein direktes Mapping zwischen dem Spielinhalt und dem theoretischen Kursmaterial und erleichtert die Wiedererkennung für Studierende, die bereits mit den Vorlesungsunterlagen vertraut sind.

Als abschließendes Element des visuellen Konzepts wurde Post-Processing eingesetzt. Um die statische Szene zu beleben und ihr Tiefe zu verleihen, ohne auf rechenintensives dynamisches Licht zurückzugreifen, wurden Farbkorrektur und Weichzeichnereffekte angewandt. Der direkte Vergleich in den Abbildungen 4.4 und 4.5 verdeutlicht diesen Effekt. Das Ergebnis ist eine warme, einladende Atmosphäre — ein Befund, der durch die Nutzertests bestätigt wurde: 6 von 12 Probanden hoben das visuelle Design explizit als

Faktor hervor, der zum angenehmen Lernerlebnis beigetragen hat.

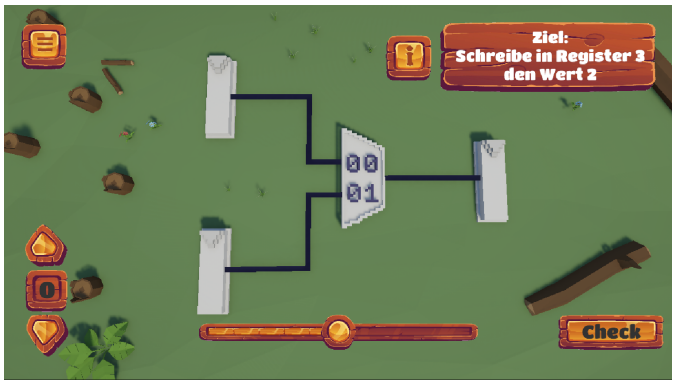


Abbildung 4.4: Darstellung der Spielszene ohne Post-Processing-Effekte



Abbildung 4.5: Finales visuelles Ergebnis nach Anwendung von Post-Processing

4.4 Mobile UI/UX Design

Bei der Gestaltung der Benutzeroberfläche bestand die zentrale Herausforderung in der Adaption komplexer Prozessorschemata an den begrenzten Bildschirmraum eines Smartphones. Das hauptsächliche Problem war es, eine Vielzahl von Knoten und Steuersignalen abzubilden, ohne dabei die sogenannte Fat-Finger-Problematik zu erzeugen [32]. Um

eine präzise Interaktion zu gewährleisten, wurden alle aktiven Steuerelemente in Übereinstimmung mit den Industriestandards konzipiert — konkret den Apple Human Interface Guidelines, nach denen die minimale Touch-Zielfläche 44×44 Punkte betragen soll.

Zur Steuerung der Informationsdichte auf dem Bildschirm wurde das Prinzip des Progressive Disclosure angewandt. Anstatt alle Funktionen gleichzeitig anzuzeigen, wurde die Oberfläche in logische Ebenen unterteilt: Sekundäre Bedienelemente sind hinter den Bildschirmrändern verborgen und werden nur bei Bedarf eingeblendet. Darüber hinaus unterstützt die Oberfläche modale Zustände: Bei Aktivierung von Lehr-Dialogen oder Hinweisen werden alle Spielelemente — mit Ausnahme der Systemmenü-Schaltfläche — automatisch ausgeblendet. Dies ermöglicht dem Spieler eine vollständige Konzentration auf den Lerninhalt und gewährleistet gleichzeitig durch die stets zugängliche Menüschaltfläche eine permanente User Control and Freedom nach Nielsen [31].

Die räumliche Orientierung innerhalb der komplexen Prozessorschemata wird durch den Einsatz eines fixierten Kamerawinkels unterstützt. Dieser erlaubt es, die Komposition jedes Levels bereits in der Entwurfsphase zu gestalten und klare visuelle Abstände zwischen Komponenten und UI-Elementen zu definieren. Für Level, deren Umfang die physischen Bildschirmgrenzen überschreitet, wurde ein System der eingeschränkten Kamerabewegung implementiert: Die Kamera bewegt sich ausschließlich entlang fester Achsen unter einem konstanten Neigungswinkel, wodurch die isometrische Perspektive erhalten bleibt und eine Desorientierung des Spielers verhindert wird. Feste Kameragrenzen (Camera Bounds) stellen sicher, dass kritische Prozessorkomponenten stets im Sichtfeld bleiben.

Ein abschließendes Gestaltungsprinzip betrifft die Informationshierarchie und die Trennung von Spiel- und Lernmodus. Die visuelle Isolation von Lehrtext durch die temporäre Ausblendung der Spieloberfläche dient als klares Signal für den Wechsel des Interaktionsmodus — von der aktiven Aufgabenlösung hin zur Theoriephase. Diese Hierarchisierung verhindert den Split-Attention-Effekt [2]: Der Studierende kann Informationen effizient verarbeiten, ohne durch die interaktiven Elemente der Simulation abgelenkt zu werden.

5 Implementierung

5.1 Technisches Grundkonzept

5.1.1 Technischer Stack

Das Projekt basiert auf der Unity Engine als zentraler Entwicklungsplattform. Die Entscheidung für Unity fiel aus mehreren Gründen. Erstens unterstützt Unity die Bereitstellung für Android, Windows, Linux, macOS und Web aus einer einzigen gemeinsamen Codebasis — ohne dass große plattformspezifische Neuimplementierungen notwendig sind (siehe Kapitel 5.1.4). Diese Multiplattformfähigkeit ist für das vorliegende Projekt besonders wichtig, da die Zielgruppe der Studierenden heterogene Gerätelandschaften mitbringt und eine breite Zugänglichkeit des Lernspiels angestrebt wird. Zweitens verfügt Unity über ein umfangreiches Ökosystem an Werkzeugen, Bibliotheken und Dokumentation, das die Entwicklung mobiler Anwendungen effizient unterstützt. Drittens brachte der Entwickler bereits fundierte Erfahrung mit der Unity-Umgebung mit, was den Einstieg beschleunigte und den Fokus auf inhaltliche und didaktische Fragestellungen lenkte, anstatt Ressourcen in das Erlernen einer neuen Plattform zu investieren.

Für die Erstellung und Bearbeitung grafischer Assets wurden GIMP (GNU Image Manipulation Program [41]) sowie die Erweiterung PhotoGIMP von Gabriel Almir [1] eingesetzt. GIMP ist ein open source Bildbearbeitungswerkzeug, das unter der GPL-Lizenz kostenlos verfügbar ist und in seinem Funktionsumfang mit proprietären Alternativen wie Adobe Photoshop vergleichbar ist. Im Kontext eines universitären Projekts ohne dediziertes Budget stellt GIMP damit die naheliegende Wahl dar: Es entstehen keine Lizenzkosten, und der Entwickler konnte auf bestehende Erfahrung mit dem Werkzeug zurückgreifen.

Die dreidimensionalen Modelle der Prozessorkomponenten wurden mit MagicaVoxel [12] erstellt — einem kostenlosen, auf den Voxel- und Low-Poly-Stil spezialisierten Modellierungswerkzeug. MagicaVoxel ermöglicht die Produktion geometrisch reduzierter 3D-Assets

ohne den Einsatz professioneller Modellierungssoftware wie Blender oder Maya, was den Produktionsprozess erheblich beschleunigt. Die inhaltliche Begründung für die Wahl des Low-Poly-Stils findet sich in Kapitel 6.1.3.

Ergänzend wurde das Asset-Paket Low-Poly Simple Nature Pack von JustCreate (Unity Asset Store, Standard EULA [22]) für dekorative Umgebungselemente eingesetzt. Der Einsatz vorgefertigter Assets für nicht-funktionale Gestaltungselemente erlaubt es, Entwicklungsressourcen gezielt auf die technisch und didaktisch relevanten Kernkomponenten zu konzentrieren. Die Entscheidung für eine naturnahe, fantasyartige Ästhetik des Umgebungsdesigns wird in Kapitel 4.3 begründet.

Auf Bibliotheksebene kommen drei zentrale Werkzeuge zum Einsatz. DOTween (HOTween v2) von Daniele Giardini [18] übernimmt die programmgesteuerte Animation von UI-Elementen und ersetzt damit den Unity Animator für Interface-Aufgaben. Dieser Ansatz reduziert die CPU-Last erheblich, da Animationen nur zum Zeitpunkt ihrer Ausführung Rechenressourcen beanspruchen — im Gegensatz zum Animator, der kontinuierlich Zustandsberechnungen durchführt. Die technische Begründung wird in Kapitel 6.2.4 ausführlich dargelegt. TextMeshPro von Unity Technologies wird für die gesamte Textdarstellung der Anwendung eingesetzt. Das zugrundeliegende SDF-Rendering (Signed Distance Field) gewährleistet eine auflösungsunabhängige, scharfe Darstellung von Schriftzeichen auf mobilen Displays unterschiedlicher Pixeldichte — ein kritischer Aspekt für die Lesbarkeit von Prozessorbeschriftungen auf kleinen Bildschirmen. Der Unity Post-Processing Stack ergänzt die visuelle Ausgabe um Farbkorrektur und atmosphärische Weichzeichnereffekte, ohne auf rechenintensives Echtzeit-Licht zurückzugreifen, was mit den Optimierungszielen für mobile Endgeräte vereinbar ist. Das Vertex-Text-Animationssystem, das für die animierten Dialoge des Charakters Charlie eingesetzt wird, basiert auf einer Eigenimplementierung unter Verwendung von Code von Scott Steffes [39] und wird in Abschnitt 5.1.2 detailliert beschrieben.

5.1.2 Softwarearchitektur

Die Gesamtarchitektur des Projekts basiert auf dem Prinzip der Trennung von Verantwortlichkeiten. Zwei voneinander unabhängige Partien kommunizieren über ein Event-System miteinander: Jede Partei betrachtet die andere als Black Box — relevant ist ausschließlich, welche Signale eintreffen und welche Reaktion darauf erwartet wird. Dieses Entwurfsmuster entspricht dem Observer-Pattern und wurde gewählt, um eine lose Kopplung zwischen den Subsystemen zu gewährleisten und die Erweiterbarkeit des Projekts zu vereinfachen.

Hauptmenü und Levelszenen
Die erste und grundlegendste Trennung besteht zwischen der Hauptmenü-Szene und den Levelszenen. Das Hauptmenü übernimmt die Rolle eines zentralen Koordinators: Es wertet den gespeicherten Spielfortschritt aus — konkret den in `PlayerPrefs` persistierten `UnlockedLevelIndex` — und aktiviert die entsprechenden Level-Auswahlschaltflächen. Beim Wechsel auf einen Level wird zunächst ein Übergangsoverlay eingeblendet, das den sogenannten Input Lag kaschiert: Da das asynchrone Laden einer neuen Szene (`SceneManager.LoadSceneAsync`) und ihre anschließende Initialisierung eine messbare Verzögerung erzeugen, würde ohne diesen Übergang für den Spieler eine sichtbare Reaktionslosigkeit entstehen. Sobald die Szene zu 90% geladen ist (`op.progress < 0.9f`), wird die Aktivierung freigegeben und das Overlay wieder ausgeblendet. Mit dem Szenenwechsel gibt das Hauptmenü die Verantwortung für den Spieler vollständig an die jeweilige Levelszene ab.

LevelManager und Regisseur
Innerhalb einer Levelszene ist die Verantwortung erneut aufgeteilt — diesmal in eine nicht-spielerische und eine spielerische Partei. Der `LevelManager` verwaltet alle Aspekte, die nicht direkt mit der Lösung des Levels zusammenhängen: das Öffnen und Schließen von Dialogmenüs, die Darstellung des Abschlussmenüs mit der Sternebewertung sowie den Übergang zur nächsten Szene. Die Kommunikation mit dem Dialogsystem erfolgt über Events — `OnDialogueBegin` und `OnDialogueEnd` — auf die der `LevelManager` beim Awake abonniert und beim Destroy wieder abmeldet, um Memory Leaks zu vermeiden.

Die spielerische Partei wird vom sogenannten Regisseur — der Klasse `BaseLevelRegisseur` und ihren Unterklassen — übernommen. Er ist verantwortlich für die Taktsteuerung, die Verwaltung des Zustandsspeichers, die Interaktion zwischen Spieler und Schaltungskomponenten sowie die Validierung der Lösung. Die genaue Funktionsweise des Regisseurs wird in Kapitel 5.3 beschrieben.

Visualisierer und Simulator
Innerhalb der spielerischen Partei besteht eine weitere interne Trennung zwischen Visualisierern und Simulator. Diese Aufteilung spiegelt direkt das Hardware-Prinzip der Trennung von Datenpfad und Steuereinheit wider.

Die Visualisierer — realisiert als Unterklassen von `BaseVizualizer`, etwa `MuiltiplexerVizualizer` — sind ausschließlich für zwei Aufgaben zuständig: die Darstellung des aktuellen Komponentenzustands für den Spieler und die Entgegennahme von Nutzereingaben. Ein Multiplexer-Visualisierer zeigt etwa die aktiven und inaktiven Eingangspfade farblich kodiert an und stellt Schaltflächen bereit, über die der Spieler den

gewünschten Pfad auswählt. Die Visualisierer besitzen dabei keine eigene Simulationslogik — sie wissen nicht, welche Konsequenzen eine Auswahl hat.

Der Simulator besteht aus einfachen, zustandsbehafteten Klassen wie `Register` oder der statischen Klasse `Multiplexer`, die die Hardware-Logik direkt abbilden. `Multiplexer` beispielsweise ist als rein statische Klasse realisiert, die ausschließlich eine Auswahlfunktion bereitstellt, wie in Listing 5.1 dargestellt ist.

Listing 5.1: Implementierung der hardwarenahen Multiplexer-Logik als statische Methode

```
1 public static int SelectNto1(int[] inputs, int control) {
2     return inputs[control];
3 }
```

Der Regisseur fungiert als Vermittler zwischen beiden Schichten: In jedem Takt liest er die aktuellen Eingaben der Visualisierer aus, übergibt sie an die Simulatorklassen und schreibt die berechneten Ergebnisse zurück an die Visualisierer. Die Abbildung 5.1 veranschaulicht diese dreischichtige Architektur.

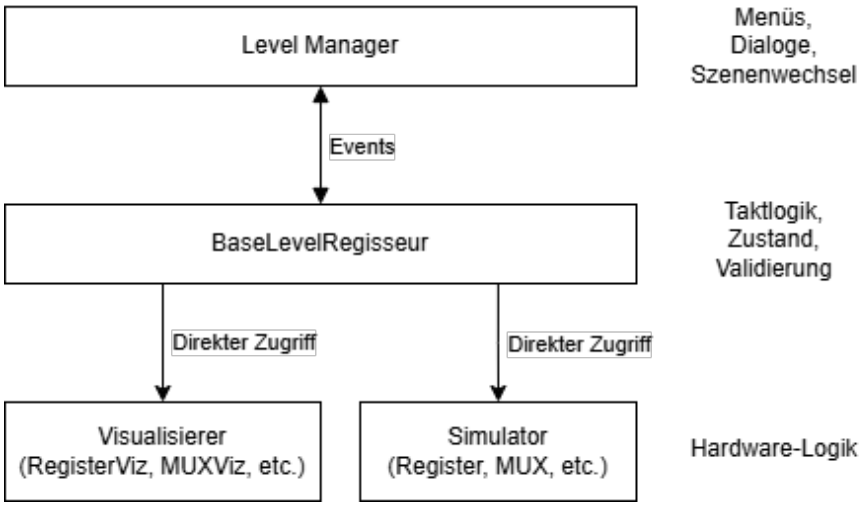


Abbildung 5.1: Schematische Übersicht der dreischichtigen Software-Architektur

Diese Architektur bietet zwei wesentliche Vorteile. Erstens ist sie beliebig erweiterbar: Ein neues Level erfordert lediglich eine neue Unterklasse des Regisseurs, die die abstrakten Methoden implementiert — die gesamte Takt- und Zustandslogik wird aus der Basisklasse

geerbt. Zweitens bleibt die Simulationslogik unabhängig von der visuellen Darstellung, was sowohl das Testen der Schaltungslogik als auch spätere UI-Änderungen vereinfacht.

5.1.3 Szenen- und Ressourcenmanagement

Szenstruktur und Ladevorgang
Das Projekt gliedert sich in eine Hauptmenü-Szene (Index 0) und eine Reihe von Levelszenen (Indizes 1–20), wobei Index 20 die wiederverwendete Basisszene des Mehrtaktprozessor-Kapitels darstellt. Alle Szenenwechsel — ob vom Menü in ein Level, zwischen Leveln oder zurück ins Hauptmenü — werden über eine einheitliche Coroutine `LoadWithTransition` abgewickelt, deren Logik in Listing 5.2 illustriert ist.

Listing 5.2: Coroutine zur asynchronen Szenensteuerung und Steuerung des Lade-Overlays

```
private IEnumerator LoadWithTransition(int targetIndex)
{
    loadingOverlay.blocksRaycasts = true;
    yield return loadingOverlay.DOFade(1f, transitionDuration).
        WaitForCompletion();

    AsyncOperation op = SceneManager.LoadSceneAsync(targetIndex);
    op.allowSceneActivation = false;

    while (op.progress < 0.9f) { yield return null; }

    op.allowSceneActivation = true;
}
```

Der Ablauf ist dabei bewusst zweiphasig gestaltet. Zunächst wird das Übergangsoverlay auf volle Deckkraft eingeblendet, um den Bildschirm zu verdecken. Erst danach wird der asynchrone Ladevorgang gestartet. Das Flag `allowSceneActivation = false` verhindert, dass Unity die neue Szene aktiviert, bevor sie zu mindestens 90% geladen ist — ein Schwellenwert, ab dem Unity intern bereit ist, die Szene zu aktivieren. Dieser Mechanismus kaschiert den sogenannten Input Lag: Der Spieler drückt eine Schaltfläche und erhält sofortige visuelle Rückmeldung durch das Overlay, während der Ladevorgang im Hintergrund abläuft.

Nach der Szenenaktivierung ist das Overlay zunächst noch vollständig sichtbar. Der `LevelManager` blendet es in `Start` mit einer kurzen Verzögerung wieder aus (siehe Listing 5.3).

Listing 5.3: Verzögerte Ausblendung des Lade-Overlays mittels DOTween-Sequenz zur Vermeidung von Rendering-Artefakten

```
...
Sequence fadeInSequence = DOTween.Sequence();
fadeInSequence.AppendInterval(0.2f);
fadeInSequence.Append(loadingOverlay.DOFade(0f, transitionDuration));
fadeInSequence.OnComplete(() => { loadingOverlay.blocksRaycasts =
    false; });
...
```

Die Verzögerung von 0,2 Sekunden gibt der Szene Zeit, alle Objekte zu initialisieren, bevor das Overlay verschwindet — so wird verhindert, dass dem Spieler ein kurzes visuelles Aufblitzen der noch nicht vollständig initialisierten Szene angezeigt wird.

Fortschrittsverwaltung
Der Spielfortschritt — konkret der Index des höchsten freigeschalteten Levels — wird über `PlayerPrefs` persistent gespeichert. Dieser Mechanismus ist plattformunabhängig und überlebt sowohl App-Neustarts als auch Geräteabstürze, da `PlayerPrefs.Save()` explizit auf den Datenträger schreibt. Beim Start des Hauptmenüs liest `MainMenu.Awake` den gespeicherten Wert aus und aktiviert die entsprechenden Level-Schaltflächen, wie in Listing 5.4 dargestelt ist.

Listing 5.4: Abruf des Spielfortschritts und Validierung der Button-Interaktivität

```
int unlockedLevels = PlayerPrefs.GetInt(UNLOCKED_LEVEL_KEY, 1);
for (int i = 0; i < levelButtons.Length; i++)
    levelButtons[i].interactable = (i < unlockedLevels);
```

Der Standardwert von 1 stellt sicher, dass beim ersten Start stets das erste Level zugänglich ist. Das Freischalten eines neuen Levels erfolgt an zwei Stellen: im `BaseLevelRegisseur` bei erfolgreicher Lösung eines Levels sowie im `LevelManager` beim expliziten Übergang zur nächsten Szene — wobei jeweils geprüft wird, ob der neue Index den bisherigen Höchstwert übersteigt, um ein unbeabsichtigtes Zurücksetzen des Fortschritts zu verhindern.

Initialisierungssequenz eines Levels
Der Ablauf beim Start eines Levels folgt einer festen Sequenz, die durch das Zusammen-

spiel von `LevelManager`, `BeginnOfLevelAnimationManager` und `DialogueManager` koordiniert wird. Unmittelbar nach der Szenenaktivierung ist das Ladeoverlay noch sichtbar und blockiert alle Eingaben. Parallel dazu abonniert der `LevelManager` in `Awake` die Events des `DialogueManager` (siehe Listing 5.5).

Listing 5.5: Kopplung zwischen DialogueManager und AnimationManager zur UI-Steuerung

```
1 ...
2 _dialogueManager.OnDialogueBegin += _animManager.
     HideUIAndShowDialogue;
3 _dialogueManager.OnDialogueEnd   += _animManager.HideDialogue;
4 _dialogueManager.OnHintEnabled   += _animManager.
     HideUIAndShowDialogue;
5 ...
```

Sobald das Overlay ausgeblendet ist, startet der `DialogueManager` automatisch den dem Level zugewiesenen Dialoggraphen. Das Event `OnDialogueBegin` veranlasst den `BeginnOfLevelAnimationManager`, die Spieloberfläche sofort auszublenden und das Dialogfenster einzublenden. Diese Umschaltung erfolgt ohne Übergangsanimation (`fadeTime = 0`), da das Ladeoverlay sie bereits verdeckt hat. Nach Abschluss des Dialogs wird `OnDialogueEnd` ausgelöst, woraufhin das Dialogfenster mit einer Slide-Down-Animation verschwindet und die Spieloberfläche mit einem Bounce-Effekt eingeblendet wird.

Der `BeginnOfLevelAnimationManager` verwaltet dabei drei unabhängige Canvas-Gruppen entsprechend der in Kapitel 2 beschriebenen Sub-Canvas-Hierarchie: die Hochfrequenz-Ebene (`oftenPanelGroup`), die Mittelfrequenz-Ebene (`mediumPanelGroup`) sowie das Dialogfenster (`dialoguePanelGroup`). Jede Gruppe besitzt eine gecachte Startposition, auf die sie nach dem Ausblenden zurückgesetzt wird — dies verhindert Positionsdrift bei wiederholten Animationsaufrufen durch `DOKill()` vor jedem neuen Tween.

Ressourcenverwaltung und Memory Management
Ein kritischer Aspekt des Szenensystems ist die korrekte Abmeldung von Event-Abonnements beim Zerstören von Objekten. Da Unity beim Szenenwechsel alle Objekte der alten Szene zerstört, können verbleibende Event-Referenzen auf bereits zerstörte Objekte zu sogenannten Ghost Calls führen — Methodenaufrufe auf nicht mehr existierende Komponenten. Sowohl `LevelManager` als auch `DialogueUI` implementieren daher `OnDestroy`-Methoden, die alle Abonnements explizit aufheben. Listing 5.6 zeigt diese Bereinigung am Beispiel des LevelManagers.

Listing 5.6: Abmeldung von Event-Abonnements zur Vermeidung von Ghost Calls

```
1 private void OnDestroy()
2 {
3     _dialogueManager.OnDialogueBegin -= _animManager.
          HideUIAndShowDialogue;
4     _dialogueManager.OnDialogueEnd   -= _animManager.HideDialogue;
5     _dialogueManager.OnHintEnabled   -= _animManager.
          HideUIAndShowDialogue;
6 }
```

Darüber hinaus werden alle laufenden DOTween-Animationen vor dem Start neuer Tweens durch `DOKill()` abgebrochen. Dies verhindert, dass konkurrierende Animationen auf dieselbe `RectTransform`-Komponente zugreifen und inkonsistente Zwischenzustände erzeugen — ein Problem, das insbesondere bei schnellen Szenenwechseln oder wiederholten Dialogaufrufen auftreten kann.

5.1.4 Plattformspezifische Unterschiede

Obwohl Unity eine gemeinsame Codebasis für alle Zielplattformen ermöglicht, erfordert die Unterstützung von Android und Windows gezielte Anpassungen sowohl auf Code-Ebene als auch in den Build-Konfigurationen. Im Rahmen dieses Projekts lassen sich diese Unterschiede in zwei Bereiche unterteilen: Eingabesteuerung und grafische Qualitätseinstellungen.

Bedingte Kompilierung
Auf Codeebene bietet Unity Präprozessor-Direktiven (`#if`, `#else`, `#endif`), die es ermöglichen, Codeabschnitte plattformabhängig ein- oder auszuschließen. Mit

```
#if UNITY_ANDROID || UNITY_IOS
```

markierte Blöcke werden bei der Windows-Kompilierung vollständig ignoriert — und umgekehrt. Dies erlaubt es, plattformspezifische Logik ohne separate Codepfade oder Projektduplikate sauber zu kapseln.

Ein konkretes Anwendungsbeispiel ist die Parallax-Effekt-Implementierung im Hauptmenü (`MenuParallax`). Auf Desktop-Plattformen wird die normalisierte Mausposition als Steuerungsinput für die Verschiebung der UI-Elemente verwendet. Auf Android steht keine Maus zur Verfügung — stattdessen wird der Gravitationssensor über das Unity Input System aktiviert und ausgelesen. Der gemessene Schwerkraftvektor wird auf den

Bereich [-1,1] begrenzt und übernimmt exakt dieselbe Rolle wie die Mausposition im Desktop-Pfad. Die Implementierung dieser Logik ist in Listing 5.7 dargestellt.

Listing 5.7: Plattformspezifische Eingabesteuerung

```
#if UNITY_ANDROID || UNITY_IOS
    // Gyroscope-basierte Steuerung
    Vector3 gravity = GravitySensor.current.gravity.ReadValue();
    _gyroPercent.x = Mathf.Clamp(gravity.x * _gyroSensitivity, -1f, 1
        f);
    Vector2 targetPos = _startPos + (_gyroPercent * _amount);
#else
    // Mausbasierte Steuerung
    Vector2 mousePercent = new Vector2(
        (mousePos.x / Screen.width) - 0.5f,
        (mousePos.y / Screen.height) - 0.5f);
    Vector2 targetPos = _startPos + (mousePercent * _amount);
#endif
```

Das Ergebnis ist ein visuell identisches Verhalten auf beiden Plattformen — realisiert durch vollständig unterschiedliche Eingabequellen.

Grafische Qualitätskonfiguration
Neben der Eingabesteuerung unterscheiden sich die Builds auf Ebene der Unity Player Settings. Mobile Endgeräte unterliegen strengen Hardware-Restriktionen hinsichtlich GPU-Leistung, Arbeitsspeicher und thermischem Verhalten — Einschränkungen, die auf Desktop-Systemen nicht in gleicher Weise relevant sind. Um eine stabile Performance auf beiden Plattformen zu gewährleisten, wurden die Qualitätsprofile plattformspezifisch konfiguriert.

Für den Android-Build wurden unter anderem folgende Maßnahmen getroffen: Die Texturkomprimierung verwendet das ASTC-Format [33], das für mobile GPUs optimiert ist und ein günstiges Verhältnis zwischen visueller Qualität und Speicherverbrauch bietet. Echtzeit-Schatten sind deaktiviert und werden durch vorberechnete Lightmaps ersetzt, wie in Kapitel 6 beschrieben. Die Ziel-Bildrate ist auf 60 FPS begrenzt, um ein kontrolliertes thermisches Verhalten zu gewährleisten.

Für den Windows-Build entfallen diese Einschränkungen weitgehend: Texturen werden in voller Auflösung ohne aggressive Komprimierung geladen, und die Anwendung kann die verfügbare GPU-Leistung des Systems ohne künstliche Begrenzung nutzen. Da beide Builds dieselbe Spielszene und dieselben Assets verwenden, ist der visuelle Unterschied

für den Endnutzer minimal — die Anpassungen betreffen ausschließlich die technische Verarbeitungsebene.

Dieses Vorgehen entspricht dem in Kapitel 1.4 definierten Rahmen der Arbeit: Beide Plattformen sind hinsichtlich ihres Funktionsumfangs identisch; die plattformspezifischen Unterschiede betreffen ausschließlich die technische Realisierung.

5.2 Datenmanagement mittels Scriptable Objects

Eine der zentralen architektonischen Entscheidungen des Projekts ist der konsequente Einsatz von Scriptable Objects (SO) für die Datenverwaltung. Scriptable Objects sind Unity-Assets, die Daten unabhängig von Szenen und MonoBehaviour-Instanzen (Objekte, die im Spielwelt instanziiert werden dürfen) speichern und per Referenz in den Arbeitsspeicher geladen werden — eine Instanz existiert genau einmal, unabhängig davon, wie viele Objekte auf sie zugreifen. Dies verhindert Datenduplizierung und gewährleistet einen stabilen Betrieb auch auf Geräten mit begrenztem Arbeitsspeicher. Im Rahmen dieses Projekts werden Scriptable Objects für drei voneinander unabhängige Systeme eingesetzt.

5.2.1 Konfiguration der Prozessor-Level

Um die Anwendungsgröße zu minimieren, wurden alle Level des abschließenden Mehrtaktprozessor - Kapitels in einer einzigen Unity-Szene realisiert. Während alle vorherigen Kapitel dem Prinzip „ein Level - eine Szene“ folgten, wird das gesamte Mehrtaktprozessor-Kapitel nach dem Prinzip „eine Szene — mehrere Datensätze“ verwaltet. Die levelspezifischen Daten sind jeweils in einem dedizierten Scriptable Object gekapselt (wie die Abbildung 5.2 darstellt) und werden zur Laufzeit dynamisch in die gemeinsame Basisszene geladen.

Das Anlegen eines neuen Levels erfordert dabei lediglich drei Schritte: das Erstellen eines neuen Knopfs im Level-Auswahlmenü, das Erstellen und Befüllen eines neuen Scriptable Objects mit den Leveldaten sowie die Verknüpfung des Knopfes mit dem ProcessorLevelLoader-Skript, dem das neue SO als Argument übergeben wird. Diese Architektur ermöglicht eine flexible und wartungsarme Erweiterung des Spiels um neue Level.

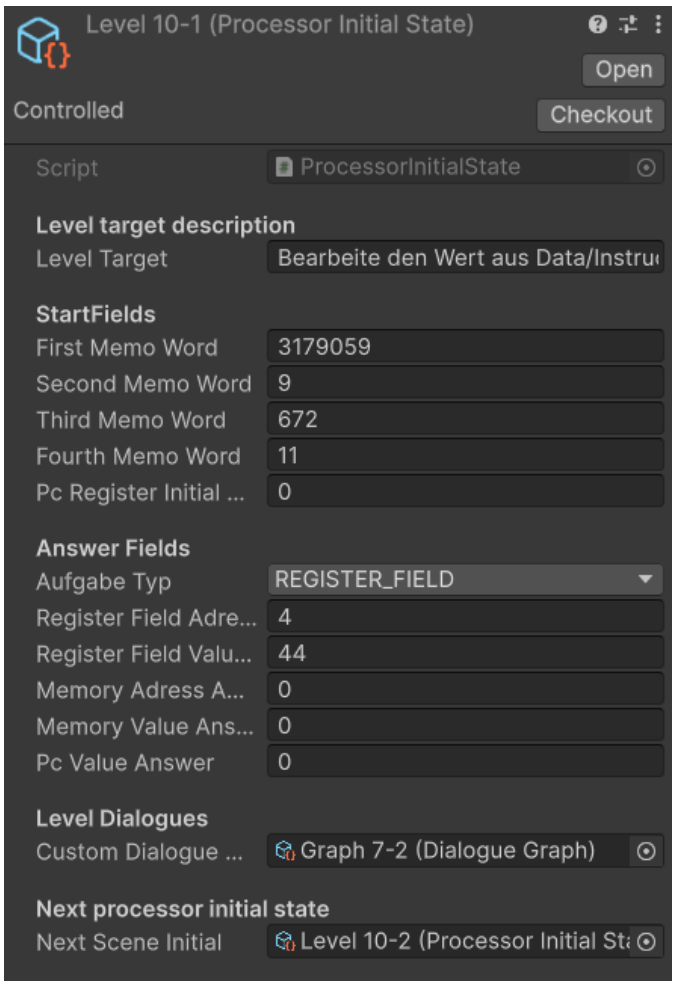


Abbildung 5.2: Beispiel eines Scriptable Objects zur Konfiguration der Level-Daten

Das Skript des Mehrtaktprozessorlevel ist eine Unterklasse der Basisklasse BaseRegisseur und besitzt eine statische Variable, die das aktuell aktive Scriptable Object hält — ein bewusst eingesetztes Singleton-Antimuster, das genau eine aktive Level-Instanz garantiert und beim Übergang zum nächsten Level automatisch aktualisiert wird. Dieses Vorgehen löst das Problem der szenübergreifenden Datenkommunikation und ermöglichte den nahtlosen Übergang vom alten zum neuen Architekturmodell.

Während der Initialisierungsphase des Levels innerhalb der Awake-Methode werden alle

relevanten Konfigurationsdaten aus dem entsprechenden Scriptable Object geladen. Dies umfasst zunächst die textuelle Aufgabenbeschreibung sowie die technischen Startparameter, namentlich den Initialwert des Program-Counter-Registers und die vier Speicherworte des Instruktionsspeichers. Darüber hinaus werden der spezifische Aufgabentyp und die für die spätere Validierung der Spielerlösung notwendigen Soll-Werte der Komponenten definiert. Den Abschluss des Datentransfers bilden der levelspezifische Dialoggraph sowie eine Referenz auf das Scriptable Object des nachfolgenden Levels, um einen nahtlosen Übergang im Spielverlauf zu gewährleisten. Ist die Referenz auf den nächsten Level null oder nicht gesetzt, wertet das System dies als Spielende und lädt das Hauptmenü. Der gewählte Aufgabentyp bestimmt dabei, welche der angegebenen Soll-Werte bei der Validierung berücksichtigt werden. Ist der Dialoggraph ungültig, wird ein Platzhalter-Dialog geladen und ein Fehler in der Unity-Konsole ausgegeben.

5.2.2 Struktur des Dialogsystems

Das Dialogsystem basiert auf einem Knotengraphen, dessen gesamte Struktur in Scriptable Objects abgebildet ist. Die grundlegende Einheit ist der Dialogknoten — ein Scriptable Object, das einen in sich abgeschlossenen Textabschnitt enthält. Mehrere solcher Knoten werden zu einem Dialoggraphen zusammengefasst, der intern als geordnete Liste realisiert ist. Der Dialog gilt als abgeschlossen, sobald der letzte Knoten erreicht wird — zu diesem Zeitpunkt werden alle während des Dialogs ausgeblendeten UI-Elemente des Levels wieder eingeblendet.

Jeder Knoten unterstützt folgende konfigurierbare Parameter: Der anzuzeigende Text kann mit Animations- und Farb-Tags dekoriert werden, um die Lesbarkeit zu verbessern und wichtige Begriffe hervorzuheben. Zudem kann für jeden Knoten eine Emotion des Charakters Charlie ausgewählt werden: Die Emotionen sind ihrerseits als Scriptable Objects realisiert, die eine Referenz auf den entsprechenden Sprite enthalten. Schließlich können Antwortoptionen konfiguriert werden, die eine Verzweigung zu einem anderen Knoten ermöglichen: Dazu genügt die Angabe des Zielindex im Graphen sowie eines Antworttextes. Sind keine Antwortoptionen konfiguriert, wird der Knoten als einfacher linearer Dialogschritt angezeigt. Die oben erwähnte Struktur ist in Abbildung 5.3 dargestellt.

Diese Struktur ermöglicht es, komplexe Dialoge mit Verzweigungen ausschließlich über den Unity-Inspektor zu erstellen, ohne Laufzeit-Parsing externer Dateiformate — was verzögerungsfreie Übergänge zwischen Dialogzeilen garantiert.

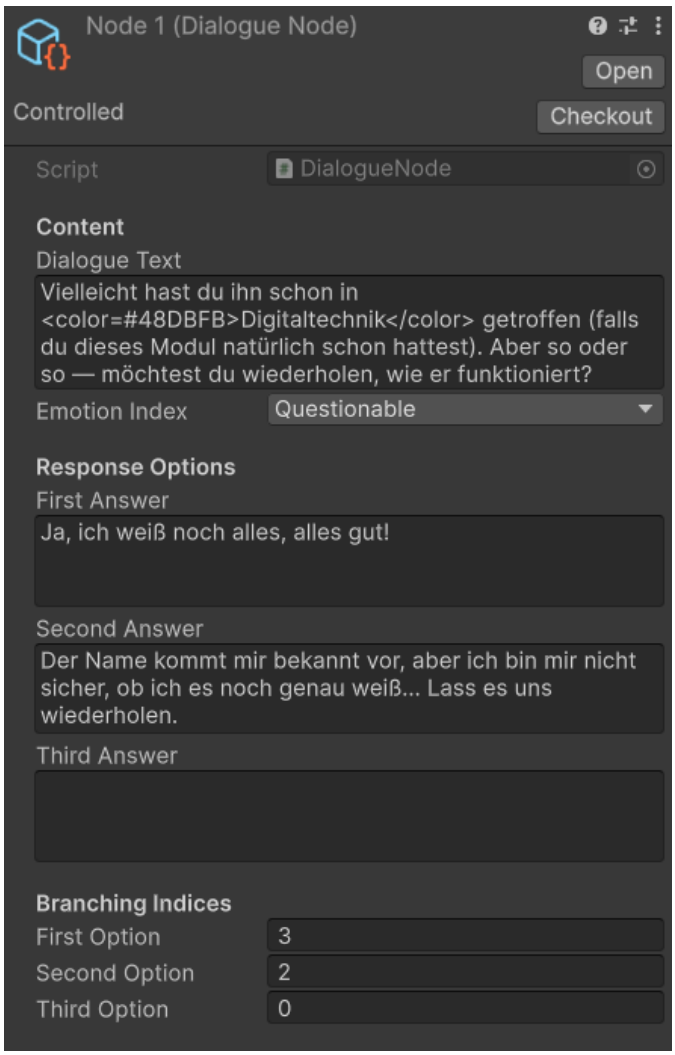


Abbildung 5.3: Datenstruktur eines Dialogknotens im Unity-Inspector

5.2.3 Verwaltung von Metadaten und Zuständen

Scriptable Objects werden im Projekt darüber hinaus für die zentrale Verwaltung kleinerer Metadaten eingesetzt, wie die Abbildungen 5.4 und 5.5 darstellen. Dazu zählen unter anderem die Konfigurationsdaten für die Charakteremotionen, die eine konsistente

Steuerung des visuellen Feedbacksystems über alle Level hinweg ermöglichen. Durch die Speicherung dieser Daten als eigenständige Assets — anstatt sie in einzelne Szenenobjekte zu kodieren — wird eine zentralisierte, wartungsarme Verwaltung des visuellen Stils der Anwendung gewährleistet.

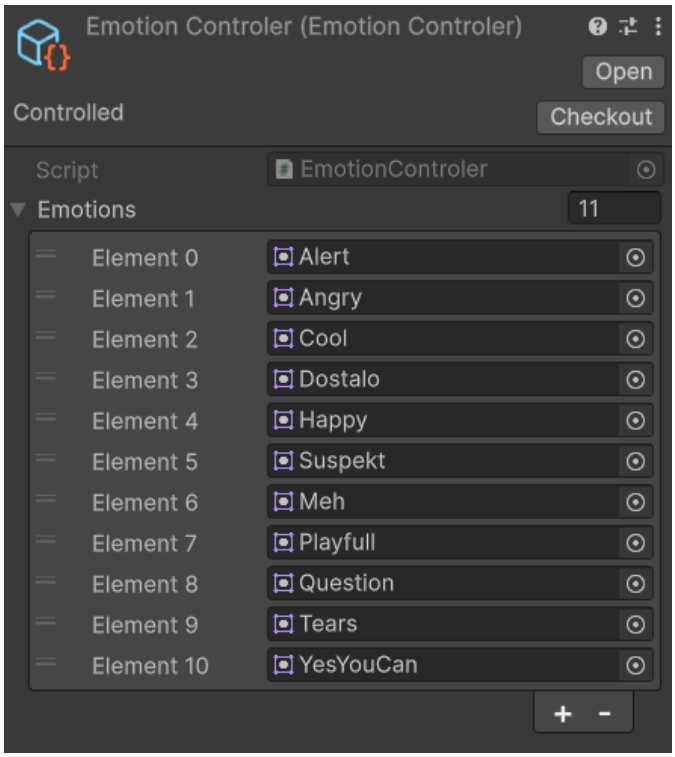


Abbildung 5.4: Struktur eines Scriptable Objects zur Definition von Charakteremotionen und den zugehörigen Sprite-Referenzen.

5.3 Simulation der Schaltungen

Der technische Kern des Projekts ist die Echtzeit-Simulation digitaler Schaltungen innerhalb der Unity-Engine. Da Unity keine nativen Werkzeuge für die Modellierung von Taktlogik oder Signalflüssen in Hardwaresystemen bietet, wurde eine eigene Simulationsarchitektur entwickelt. Ihre Grundlage bildet die abstrakte Basisklasse `BaseLevelRegisseur`,



Abbildung 5.5: Implementierung der Prozessorzustände (FSM) als Scriptable Objects zur Verwendung innerhalb des Hinweissystems.

von der alle levelspezifischen Steuerklassen erben. Diese Architektur trennt die allgemeine Takt- und Zustandslogik konsequent von der levelspezifischen Implementierung.

5.3.1 Taktlogik und Zustandsverwaltung

Das zentrale didaktische Merkmal des Spiels ist die taktgenaue, schrittweise Ausführung von Schaltungen — mit der Möglichkeit, beliebig viele Takte zurückzuspringen. Die Implementierung dieser Mechanik basiert auf zwei Coroutines: `NextTickSequence` und `PrevTickSequence`.

Beim Vorwärtsschritt (`NextTickSequence`) läuft folgende Sequenz ab: Zunächst werden alle interaktiven UI-Elemente blockiert, um inkonsistente Nutzereingaben während der Visualisierung zu verhindern. Anschließend wird `BlinkClockedComponents` aufgerufen, das alle getakteten Komponenten der Schaltung visuell aufleuchten lässt — analog zur steigenden Taktflanke in realer Hardware. Danach wartet die Coroutine auf den Abschluss von `RunBusVisualizations`, das die animierten Signalflüsse auf den Datenbussen darstellt. Erst nach Abschluss dieser Visualisierung wird `HandleClockUpdate` aufgerufen,

das die eigentliche Logik des Taktes ausführt und die internen Komponentenwerte aktualisiert. Abschließend wird der aktuelle Zustand im Zustandsspeicher abgelegt und der Takt-Counter inkrementiert. Dieser Vorgang wird Schritt für Schritt im Listing 5.8 dargestellt.

Listing 5.8: Ablauf einer Taktschritt-Sequenz innerhalb der NextTickSequence-Coroutine

```
NextTickSequence:
  BlockIngameInteractables()       // Blockieren von Nutzereingabe
  BlinkClockedComponents()         // Start der Animation
     gekennzeichnen
  yield RunBusVisualizations()     // Auf Ende der Animation warten
  HandleClockUpdate()              // Logik berechnen
  tickCounter++
  saveCurrentStateAt(tickCounter)  // Speichern des Zustandes
  ReleaseIngameInteractables()
```

Diese Reihenfolge ist didaktisch bewusst gewählt: Der Spieler sieht zuerst die Signale fließen und beobachtet dann die Konsequenz im Zustand der Komponenten — analog zur tatsächlichen Kausalität in realen Schaltungen.

Der Zustandsspeicher ist als Array `_tickStateValues` realisiert, dessen Länge durch `_maxTickNumber` definiert wird. In jedem Takt wird der aktuelle Zustand über die abstrakte Methode `GetCurrentState` als generisches Objekt erfasst und im Array abgelegt. In der konkreten Implementierung — etwa in `LevelZeroRegisseur` — wird der Zustand als `struct` abgebildet, der alle relevanten Komponentenwerte enthält (siehe Listing 5.9).

Listing 5.9: Definition einer Datenstruktur zur Kapselung des Systemzustands für einen spezifischen Prozessor-Level

```
public struct LevelZeroState
{
    public int RegisterAValue;
    public int RegisterBValue;
    public int OutputRegisterValue;
    public bool RegisterAWE;
    public bool RegisterBWE;
    public bool OutputRegisterWE;
}
```

Beim Rückwärtsschritt (`PrevTickSequence`) wird der Takt-Counter dekrementiert, `ReverseBusVisualizations` gestartet — das die Busanimation in umgekehrter Richtung abspielt — und anschließend der gespeicherte Zustand über `ApplyState` auf alle

Komponenten zurückgeschrieben. Das Ergebnis ist ein vollständiger Rollback auf den exakten Systemzustand des vorherigen Taktes. Es sei dabei ausdrücklich darauf hingewiesen, dass dieses Zurückspulen in realen digitalen Schaltungen nicht möglich ist; es handelt sich um eine rein didaktische Abstraktion, die dem Spieler ein experimentelles, fehlertolerantes Erkundungsverhalten ermöglicht.

5.3.2 Signalfluss und Komponentenkommunikation

Die Ausführungslogik eines einzelnen Taktes ist in der abstrakten Methode `HandleClockUpdate` gekapselt, die in jeder levelspezifischen Klasse individuell implementiert wird. Am Beispiel von `LevelZeroRegisseur` lässt sich das grundlegende Muster veranschaulichen, welches in Listing 5.10 dargestellt wird.

Listing 5.10: Beispiel für eine Implementierung der Schaltungslogik und Signalsynchronisation

```
protected override void HandleClockUpdate()
{
    srcA.WriteEnable = _registerSrcAVisualizer.isWriteEnabled;
    srcB.WriteEnable = _registerSrcBVisualizer.isWriteEnabled;
    output.WriteEnable = _registerOutputVisualizer.isWriteEnabled;

    srcB.Input = srcA.Output;
    output.Input = srcB.Output;

    srcA.PreClockUpdate();
    srcB.PreClockUpdate();
    output.PreClockUpdate();

    srcA.Clock();
    srcB.Clock();
    output.Clock();
}
```

Der Ablauf spiegelt das Hardware-Prinzip direkt wider. Zunächst werden die WriteEnable-Signale der logischen Komponenten mit dem aktuellen Zustand der UI-Visualisierer synchronisiert — das heißt, die Entscheidungen des Spielers werden in die Simulationslogik übertragen. Anschließend werden die Signalverbindungen zwischen den Komponenten hergestellt: Der Ausgang von `srcA` wird als Eingang von `srcB` gesetzt, dessen Ausgang wiederum als Eingang von `output`. Danach wird für alle Komponenten zunächst

PreClockUpdate aufgerufen, das Vorberechnungen durchführt, und abschließend Clock, das den neuen Wert tatsächlich übernimmt — sofern WriteEnable aktiv ist.

Parallel zur Logik läuft die visuelle Darstellung des Signalflusses über den BusController. Die Methode RunBusVisualizations steuert, welche Bussegmente in welcher Reihenfolge animiert werden, und übergibt dabei den tatsächlichen Signalwert, der auf dem jeweiligen Bus transportiert wird. Die Coroutine wartet mit

```
yield return new WaitUntil(() => _busController.NoActiveSignals)
```

auf den Abschluss aller aktiven Animationen, bevor die Taktlogik ausgeführt wird. Dadurch ist sichergestellt, dass der Spieler den Signalfluss vollständig beobachten kann, bevor sich der Zustand der Komponenten ändert.

5.3.3 Validierung der Spielerlösung

Die Überprüfung der Spielerlösung erfolgt über die Methode CheckSolution, die in BaseLevelRegisseur implementiert ist und intern die abstrakte Methode CheckWinCondition aufruft. Diese Trennung ermöglicht es, die allgemeine Feedback- und Fortschrittslogik zentral zu verwalten, während die eigentliche Gewinnbedingung levelspezifisch definiert wird.

Liefert CheckWinCondition den Wert true, gilt das Level als gelöst. Das System ermittelt daraufhin die Anzahl der verdienten Sterne anhand der kumulierten Fehlversuche (falledTries) und vergleicht sie mit drei konfigurierbaren Schwellenwerten (_threeStarCondition, _twoStarCondition, _oneStarCondition). Das Ergebnis wird an den LevelManager übergeben, der das Abschluss-Menü öffnet und den nächsten Level freischaltet. Die Freischaltung erfolgt über PlayerPrefs, wobei der höchste freigeschaltete Szenenindex persistent gespeichert wird.

Ist die Lösung hingegen inkorrekt, wird der Fehlversuchszähler inkrementiert und eine animierte Fehlerindikation ausgelöst: Ein Kreuz-Symbol erscheint mit einem Bounce-Effekt und einer Schüttelanimation (realisiert über DOTween) und verschwindet nach kurzer Verzögerung wieder. Dieses unmittelbare visuelle Feedback entspricht dem in Kapitel 4.2 beschriebenen Prinzip des formativen Feedbacks — der Spieler erhält sofortige Rückmeldung, ohne den Spielfluss zu unterbrechen.

5.4 Distribution und Deployment

Obwohl das primäre Ziel dieser Arbeit in der Konzeption und Implementierung des Lernspiels bestand und eine breite Distribution nicht Teil der ursprünglichen Zielsetzung war, wurde im Verlauf des Projekts entschieden, die Anwendung auf etablierten Distributionsplattformen zu veröffentlichen. Dies dient der Zugänglichkeit für interessierte Leser sowie der praktischen Nachvollziehbarkeit der in dieser Arbeit beschriebenen Ergebnisse. Die App ist über die unten genannten Quellen verfügbar.

Google Play Store
Der Google Play Store stellt die primäre Distributionsplattform für die Android-Version des Projekts dar und wurde gewählt, da er mit über drei Milliarden aktiven Geräten die reichweitenstärkste Plattform für Android-Anwendungen bildet.

Der Prozess der Veröffentlichung erwies sich als deutlich aufwändiger als initial erwartet. Die Vorbereitung des Projekts — einschließlich der korrekten Metadaten, der Einhaltung aller Plattformrichtlinien sowie der grafischen und textlichen Gestaltung der Store-Präsenz — nahm insgesamt mehr als drei Wochen in Anspruch. Ein zentrales Hindernis stellte dabei die obligatorische Phase des geschlossenen Testens dar, die Google vor der Freigabe zur öffentlichen Veröffentlichung vorschreibt. Diese Phase erfordert eine Fokusgruppe von 12 bis 20 Personen, die die Anwendung über einen Zeitraum von mindestens zwei Wochen auf ihren Geräten installiert halten und regelmäßig testen.

Im Rahmen dieses Prozesses wurde eine Vielzahl von Nutzerfeedback gesammelt. Da dieses Feedback jedoch erst nach dem Abschluss der in Kapitel 7 beschriebenen Evaluation eingegangen ist, wird es im Rahmen dieser Arbeit nicht weiter ausgewertet und soll in zukünftigen Iterationen des Projekts Berücksichtigung finden.

Ein zusätzlicher Vorteil der Google-Play-Veröffentlichung ist die automatische Verfügbarkeit der Anwendung über Google Play for PC — eine Funktion, die es ermöglicht, Android-Anwendungen direkt auf Windows-Rechnern auszuführen (siehe Tabelle 5.1).

Dies vereinfacht den Zugang für Studierende, die kein Android-Gerät besitzen, ohne dass eine separate Windows-Build gepflegt werden muss, und steht im Einklang mit den in Kapitel 1.4 definierten Rahmenbedingungen dieser Arbeit.

itch.io
Als ergänzende Plattform wurde itch.io gewählt, die im Indie-Spielebereich weit verbreitet und für ihre niedrige Einstiegshürde bekannt ist: Inhalte können ohne Moderationsprozess

oder Qualitätsprüfung veröffentlicht werden. Auf dieser Plattform werden die Windows-Version sowie zukünftige Builds für Linux und macOS bereitgestellt. Damit dient itch.io als Fallback-Kanal für Nutzer, bei denen der Google Play Store nicht verfügbar ist oder nicht genutzt werden kann.

GitHub
Der vollständige Quellcode des Projekts ist öffentlich auf GitHub zugänglich (siehe Tabelle 5.1). Im Bereich Releases des Repositories sind vorkompilierte, direkt ausführbare Builds für Android und Windows verfügbar. Darüber hinaus enthält das Repository eine Anleitung zur eigenständigen Kompilierung des Projekts aus dem Quellcode. Der jeweils aktuellste Entwicklungsstand des Projekts befindet sich im Branch `main`, beim Commit `17a5d66`. Alle nachfolgenden Commits gehören zur Weiterentwicklung des Projekts und sind nicht Bestandteil dieser Arbeit, da das Projekt auch nach Abgabe aktiv weiterentwickelt werden soll (siehe Kapitel 8.2).

Einige Kommentare zum Code sind auf Russisch erfasst, da es die Muttersprache des Entwicklers ist und für Notizen besser war. Dazu sind gemischte Kommentare auf Englisch und Deutsch zu finden, da Projekt keine einheitliche Strategie über Kommentare folgte und für Codeabschnitten die Sprache verwendet wurde, deren Wortschatz im Moment am passenden in Augen des Entwicklers war.

Ein Teil der Kommentare im Quellcode ist auf Russisch verfasst, da es die Muttersprache des Entwicklers ist. Weitere Kommentare sind auf Englisch und Deutsch gehalten, da das Projekt keine einheitliche Kommentarstrategie verfolgte. Darüber hinaus sind nicht alle Codebereiche gleich ausführlich kommentiert, da eine vollständige Kommentierung zwar in der ersten Phase des Projekts angestrebt wurde, aufgrund des Umfangs jedoch nicht konsequent fortgeführt werden konnte.

Tabelle 5.1: Links zur Projektdistribution

Plattform	**Link**
Google Play Store	`https://play.google.com/store/apps/details?id=com.edu.mehrtaktproz.sim&pcampaignid=web_share`
itch.io	`https://ggalya.itch.io/mehrtakt-abenteuer?secret=ESutD7JerurFW9SCPGtlfIL0FQ8`
GitHub	`https://github.com/GGalya1/Bachelor-project-at-TU-Darmstadt.git`
Github Releases	`https://github.com/GGalya1/Bachelor-project-at-TU-Darmstadt/releases/tag/v0.7.6`

6 Optimierung

Die Qualität eines Lernspiels hängt nicht allein von seinen didaktischen Inhalten ab, sondern maßgeblich von der technischen Stabilität der Anwendung. Unterbrechungen des Spielflusses durch Bildratenschwankungen oder verzögerte Reaktionen der Benutzeroberfläche zerstören den Flow-Zustand und beeinträchtigen damit direkt die kognitive Verarbeitung der Lerninhalte [9]. Vor diesem Hintergrund wurde die Optimierung in zwei komplementäre Bereiche unterteilt: die grafische Optimierung, die die Last auf den Grafikprozessor reduziert, und die Rechenoptimierung, die einen effizienten Einsatz des Hauptprozessors gewährleistet. Beide Maßnahmen verfolgen dasselbe übergeordnete Ziel: die Schaffung einer technisch stabilen Umgebung, in der sich die Lernenden vollständig auf die Logik des Mehrtaktprozessors konzentrieren können.

Die gewählte Entwicklungsplattform, das Game-Engine Unity, zeichnet sich durch eine hohe Flexibilität aus und unterstützt eine effiziente Multiplattform-Bereitstellung. Dennoch bieten die automatisierten Werkzeuge der Engine keine erschöpfende Optimierung für spezifische Hardwarespezifikationen. Im Rahmen dieser Arbeit, die auf mobile Endgeräte abzielt, ist die Berücksichtigung von Hardware-Restriktionen ein kritischer Aspekt.

Mobile Plattformen sind durch eine begrenzte Speicherbandbreite, strikte Energieverbrauchslimits und eine Neigung zu thermischem Throttling bei intensiven GPU-Berechnungen gekennzeichnet. Da im Entwicklungsplan keine Phase für ein umfassendes plattformübergreifendes Benchmarking an einer repräsentativen Stichprobe von Geräten vorgesehen war, wurde die Entscheidung getroffen, eine präventive Optimierung zu implementieren. Das Ziel besteht darin, eine stabile Bildwiederholrate (FPS) auch auf Geräten des Budgetsegments und älteren Modellen zu gewährleisten.

Die technischen Erläuterungen und Implementierungsstrategien dieser Arbeit, insbesondere in den Abschnitten 6.1 und 6.2, basieren maßgeblich auf den offiziellen Ressourcen und Best-Practice-Leitfäden von Unity Technologies. Das Unternehmen stellt ein umfangreiches Spektrum an Lehrmaterialien zur Verfügung, das von spezialisierten Lernprojekten bis hin zu vertiefenden Fachbüchern und Kursen reicht. Um die Lesbarkeit zu fördern und

redundante Quellenangaben innerhalb der technischen Detailbeschreibung zu vermeiden, stützen sich die dort verwendeten Begriffe, Algorithmen und Architekturansätze auf die in Tabelle 6.1 aufgeführte Fachliteratur.

Tabelle 6.1: Zusammenfassung der verwendeten Best-Practice-Leitfäden von Unity Technologies

Kategorie	**Titel**	**Beschreibung**
Projektübergreifend	*Ultimate guide to profiling Unity games* [48]	Analyse von Performance-Engpässen und Nutzung von Profiling-Tools.
Projektübergreifend	*Optimize your game performance for mobile, XR, and the web* [46]	Spezifische Optimierungstechniken für hardwarebeschränkte mobile Plattformen.
Kapitel 6.1	*UI Toolkit for advanced Unity developers* [47]	Architektur und Optimierung komplexer Benutzeroberflächen.
Kapitel 6.2	*Create modular game architecture in Unity with ScriptableObjects* [43]	Entkopplung von Systemen für eine effiziente Datenverwaltung.
Kapitel 6.2	*Level up your code with design patterns and SOLID* [45]	Anwendung von Software-Entwurfsmustern zur Steigerung der Code-Effizienz.
Kapitel 6.2	*Introduction to DOTS concepts, features, & samples* [44]	Datenorientierte Programmierung zur Maximierung der Rechenleistung.

6.1 Grafische Optimierung

Die grafische Optimierung in diesem Projekt stellt einen Kompromiss zwischen der visuellen Ästhetik und dem damit verbundenen Rechenaufwand dar. Das Hauptaugenmerk lag auf der Reduzierung der Belastung des Grafikprozessors durch die Optimierung von Beleuchtung, Geometrie und Benutzeroberfläche.

6.1.1 Schatten und Beleuchtung

In der modernen Computergrafik gehört die Berechnung von Beleuchtung und Schatten in Echtzeit zu den rechenintensivsten Operationen. Für jedes Einzelbild (wovon ein mobiles Spiel mindestens 60 pro Sekunde liefern sollte) muss der Grafikchip die Trajektorien der Lichtstrahlen, deren Kollision mit Objekten und die Dichte der erzeugten Schatten kontinuierlich neu berechnen.

Für mobile Endgeräte ergeben sich aus dieser hohen Rechenlast zwei kritische Herausforderungen: Thermal Throttling und ein erhöhter Energieverbrauch. Das Thermal Throttling beschreibt dabei den Prozess, bei dem intensive Berechnungen zur Überhitzung des Geräts führen, was eine erzwungene Leistungsreduzierung und damit verbundene, abrupte Performance-Einbrüche zur Folge hat. Parallel dazu führt die konstante Belastung des Grafikchips zu einer schnellen Entladung des Akkus. Dies ist insbesondere im Kontext einer Lernanwendung problematisch, da deren Nutzung auf längere Zeiträume ausgelegt ist und eine unzureichende Akkulaufzeit die praktische Nutzbarkeit im Lernalltag erheblich einschränken würde.

Um ein Gleichgewicht zwischen Bildqualität und Energieeffizienz zu erreichen, wurde die Technologie des Baked Global Illumination angewandt. Der Kern dieser Methode besteht darin, die Beleuchtung und Schatten bereits während der Entwicklungsphase vorab zu berechnen. Die Ergebnisse dieser Berechnungen werden in speziellen Texturen – den sogenannten Lightmaps – gespeichert, die während der Laufzeit über die Objekte gelegt werden. Dies ermöglichte einerseits die vollständige Eliminierung von GPU-Kosten für die Schattenberechnung in Echtzeit. Gleichzeitig erlaubt dieser Ansatz die Implementierung von weichen Schatten und indirekter Beleuchtung, welche auf mobilen Plattformen unter Echtzeitbedingungen normalerweise nicht realisierbar wären. Zudem konnte durch den Einsatz von gebackener Ambient Occlusion die visuelle Tiefe der Prozessorszene signifikant erhöht werden, ohne FPS negativ zu beeinflussen.

6.1.2 Kamera und Objekte auf der Szene

Bei der Gestaltung des dreidimensionalen Raums besteht das Hauptproblem im Konflikt zwischen der Notwendigkeit einer detaillierten Umgebung zur Aufrechterhaltung der Immersion und den begrenzten Kapazitäten mobiler Grafikchips. Jedes Objekt in der Szene erfordert eine bestimmte Anzahl von Zeichenbefehlen, was die Flüssigkeit der Anwendung direkt beeinflusst. Um die Systemlast zu minimieren, ohne die visuelle Qualität

zu beeinträchtigen, wurde eine spezifische Strategie für die virtuelle Kamera und die Objektdarstellung entwickelt.

Als primäre Lösung wurde eine statische Kamera bzw. eine Kamera mit einem fixierten Betrachtungswinkel gewählt. Im pädagogischen Kontext ist diese Entscheidung dadurch begründet, dass der Fokus des Lernenden konstant auf den logischen Einheiten des Prozessors liegen sollte, ohne durch die Kamerasteuerung abgelenkt zu werden. Aus technischer Sicht erlaubt die fixierte Kameraposition eine erhebliche Optimierung der geometrischen Komplexität. Da der Nutzer die Objekte nur aus einer Richtung betrachtet (Top-Down-Perspektive), ist es möglich, nicht sichtbare Flächen der Modelle zu entfernen. Dies reduziert die Anzahl der zu verarbeitende Polygone erheblich und setzt CPU-Ressourcen für die Spiellogik frei. Falls die Prozessorarchitektur nicht auf einen einzigen Bildschirm passt, bewegt sich die Kamera entlang der X- und Y-Achsen, behält dabei jedoch den fest vorgegebenen Neigungswinkel bei. Dieser Ansatz erlaubt den effizienten Einsatz von Frustum Culling – dem automatischen Ausschluss von Objekten außerhalb der Sichtpyramide der Kamera. Dadurch wird sichergestellt, dass Systemressourcen nur für die Elemente aufgewendet werden, die für den Nutzer im aktuellen Moment sichtbar sind.

Eine weitere Ebene der Optimierung betrifft die visuelle Gestaltung der Szenenbegrenzungen. Standardmäßig verwendet die Unity-Engine eine Skybox — eine Panoramatextur, die den Himmel und die Umgebung in einer 360-Grad-Ansicht simuliert. Für ein Spiel mit einer Top-Down-Perspektive ist die Nutzung dieser Technologie jedoch redundant und ineffizient. Da der „Himmel“ physisch nicht in das Sichtfeld der Kamera gelangen kann, erzeugt die Berechnung seiner Texturen und der damit verbundenen Beleuchtung eine unnötige Belastung für den Videospeicher. Darüber hinaus muss bei der Entwicklung für mobile Plattformen die Vielfalt der Bildschirmformate berücksichtigt werden. Auf Geräten mit breitem Display können an den Rändern der Szene leere Bereiche entstehen, die üblicherweise durch zusätzliche dekorative Modelle abgedeckt werden müssten, was wiederum die Anzahl der Draw Calls erhöhen würde.

Zur Lösung dieses Problems wurde die Methode Solid Color angewandt. Anstatt einen komplexen Hintergrund zu rendern, füllt die Kamera alle leeren Bereiche mit einer einheitlichen Farbe, die programmgesteuert mit der Farbe des Bodens oder der Prozessor-Grundplatte synchronisiert ist. Wie in Abbildung 6.1 dargestellt, erzeugt die Verwendung des Standardhintergrunds mit einer Skybox visuelle Lücken an den Bildschirmrändern, was zusätzliche Ressourcen für deren Maskierung erfordern würde.

Abbildung 6.2 zeigt die endgültige Lösung, bei der durch die Nutzung von Solid Color der Effekt eines nahtlosen, „unendlichen“ Raums erzielt wird. Dies erspart nicht nur die Modellierung einer unnötigen Umgebung, sondern minimiert auch den Effekt der



Abbildung 6.1: Darstellung der sichtbaren Modellgrenzen und Hintergrundartefakte bei Verwendung einer herkömmlichen Skybox

Überzeichnung und gewährleistet so eine stabile Performance auf mobilen Endgeräten mit jeder beliebigen Bildschirmauflösung.



Abbildung 6.2: Optimierte Hintergrunddarstellung durch Anwendung der Solid-Color-Methode zur Erzeugung eines nahtlosen Übergangs

6.1.3 Objektoptimierung

Die Effizienz des Renderings dreidimensionaler Objekte hängt unmittelbar von ihrer geometrischen Komplexität und der Art der Datenverwaltung innerhalb der Engine ab. Im Rahmen der Entwicklung des Lernspiels zur Prozessorarchitektur enthält die Szene eine Vielzahl repetitiver Elemente, wie logische Gatter, Register und Datenbusse. Um die Last auf den CPU und den GPU zu minimieren, wurde eine Optimierung der Meshes (Polygonnetze) und Materialien durchgeführt.

Der erste Schritt der Optimierung bestand in der Wahl des künstlerischen Stils Low Poly. Im Kontext einer ingenieurwissenschaftlichen Aufgabe ist diese Entscheidung in erster Linie technischer Natur: Sie ermöglicht eine radikale Reduzierung der Scheitelpunktanzahl (Vertex Count) und der Polygone. Da mobile Grafikchips über ein begrenztes Budget für die Geometrieverarbeitung pro Frame verfügen, verhindert der Einsatz vereinfachter Modelle eine Überlastung der Grafik-Pipeline. Alle 3D-Modelle im Projekt wurden von redundanter Geometrie und unsichtbaren inneren Flächen bereinigt, wie die Abbilung 6.3 zeigt. Dies ermöglichte es, Rechenressourcen für wichtigere Aufgaben freizusetzen, wie etwa die dynamische Visualisierung von Datenflüssen innerhalb des Mehrtaktprozessors, die eine hohe Animationsflüssigkeit erfordert.

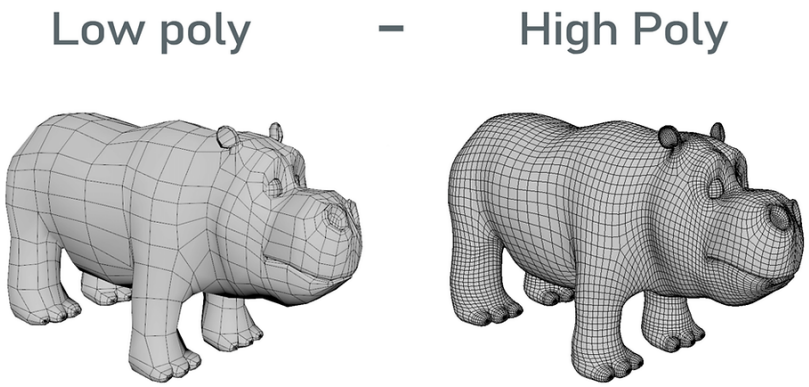


Abbildung 6.3: Vergleich zwischen einer unoptimierten High-Poly-Geometrie und einem Low-Poly-Modell. Die Reduktion der Polygondichte minimiert die Rechenlast ohne signifikanten Verlust der visuellen Qualität im Anwendungskontext (Bildquelle: `https://www.taangastudios.com/post/what-are-low-poly-and-high-poly-3d-models`)

Ein zentrales Werkzeug für die Objektverwaltung im Projekt sind Prefabs. In der Unity-Umgebung ist ein Prefab eine Vorlage oder eine Master-Kopie eines Objekts, die mehrfach in der Szene verwendet werden kann. Aus Sicht der Optimierung stellt die Verwendung von Prefabs eine effiziente Methode der Speicherverwaltung dar. Anstatt Informationen über jeden einzelnen Logikbaustein als neues, einzigartiges Objekt in den Arbeitsspeicher zu laden, nutzt die Engine den Mechanismus des Data Sharing. Alle Kopien eines Prefabs verweisen auf dieselben Geometrie- und Materialdaten. Darüber hinaus ermöglichte dies den Einsatz der GPU Instancing-Technologie, wodurch die Grafikkarte eine Vielzahl identischer Objekte innerhalb eines einzigen Zeichenbefehls rendern kann. Dies ist für die Aufrechterhaltung einer hohen Bildwiederholrate von entscheidender Bedeutung.

Zusätzliche Optimierungen wurden durch die Bearbeitung von Materialien und Texturen erreicht. Für mobile Plattformen stellt der häufige Wechsel von Texturen während des Renderings eine ressourcenintensive Operation dar, die zu Verzögerungen führen kann. Um dieses Problem zu lösen, wurde die Technik des Texture Atlas angewandt. Anstatt zahlreicher kleiner Texturdateien für verschiedene Objekte wurden alle notwendigen Bilder in einer einzigen großen Texturmap zusammengefasst. In Verbindung mit modernen Kompressionsalgorithmen (wie ASTC [33]) reduzierte dies nicht nur das Gesamtgewicht der Anwendung, sondern verkürzte auch die Ladezeiten der Szenen und gewährleistete so einen schnellen Zugriff auf die Lerninhalte. Somit ermöglichte der systemische Ansatz bei der Objektoptimierung die Schaffung einer visuell ansprechenden und technisch stabilen Umgebung für die Simulation des Prozessors. Die resultierende Performance-Stabilität ist dabei nicht nur ein technischer Vorteil, sondern eine didaktische Voraussetzung: Flüssige Animationen der Datenflüsse zwischen den Prozessorkomponenten sind für die korrekte kognitive Verarbeitung der Signalwege essenziell [40].

6.1.4 Optimierung der Benutzeroberfläche

In der Unity-Umgebung erfordert das UGUI-System eine strikte Kontrolle, da Architekturfehler innerhalb dieses Systems zu Verzögerungen führen können, die für eine Bildungsanwendung kritisch und unzulässig sind. Jegliche technischen Ruckler der Benutzeroberfläche zerstören den Zustand des Flows und hindern den Nutzer daran, sich auf die Lösung logischer Aufgaben zu konzentrieren. Um einen Konzentrationsverlust des Spielers zu vermeiden und ein hohes Maß an Immersion aufrechterhalten zu können, wurde Benutzeroberfläche optimiert. Diese zielte darauf ab, die Belastung des Hauptprozessors zu minimieren und eine sofortige Systemreaktion zu gewährleisten.

Batching
Eine der ressourcenintensivsten Aufgaben beim Rendering der Benutzeroberfläche ist die Organisation der Interaktion zwischen dem CPU und dem Grafikprozessor GPU. Im Standardmodus erfordert jedes einzelne grafische Element – ob Schaltfläche, Textfeld oder Signal-Icon – einen individuellen Zeichenbefehl, den sogenannten Draw Call. Das Problem besteht darin, dass jeder dieser Aufrufe einen erheblichen Kommunikations-Overhead verursacht: Der Prozessor benötigt Zeit, um die Daten vorzubereiten und den Zustand der Grafikkarte umzuschalten.

In einem Lernspiel zur Prozessorarchitektur können sich gleichzeitig mehrere Dutzend Indikatoren auf dem Bildschirm befinden, die den aktuellen Zustand von Registern und Datenbussen anzeigen. Ohne Optimierung könnte die Gesamtzahl der Draw Calls den kritischen Schwellenwert der Bandbreite des mobilen Systems überschreiten, was unweigerlich zu einem Abfall der Bildrate und einer Überhitzung des Geräts führen würde.

Zur Minimierung dieser Last wurde der Mechanismus des UI Batching angewandt. Der Kern dieser Technologie besteht in der automatischen Gruppierung mehrerer grafischer Elemente zu einem einzigen Paket (Batch), um diese innerhalb eines Zyklus gleichzeitig an die Grafikkarte zu übertragen. Technisch gesehen ist die Hauptbedingung für eine erfolgreiche Zusammenführung der Elemente die Verwendung identischer Materialien und einer gemeinsamen Textur. Um diese Bedingung zu erfüllen, wurden alle visuellen Assets der Benutzeroberfläche in einem Sprite Atlas zusammengefasst – einem speziellen Container, der eine einzige große Texturmap darstellt und alle kleinen Icons sowie Gestaltungselemente enthält.

Durch die Verwendung von Atlases nimmt die Unity-Engine vereinzelte UI-Elemente als Teile einer einzigen grafischen Ressource wahr. Dies ermöglichte es, die Anzahl der Zeichenbefehle um ein Vielfaches zu reduzieren. Infolgedessen wurde eine stabil hohe Performance erreicht, selbst in Momenten intensiver Visualisierung des Mehrtaktprozessors, in denen das System eine Vielzahl von Indikatoren und Datenflüssen ohne für den Nutzer sichtbare Verzögerungen aktualisieren muss. In einem Bildungskontext ist diese Stabilität unmittelbar lernrelevant: Jeder sichtbare Ruckler beim Aktualisieren der Prozessorindikatoren unterbricht den kognitiven Verarbeitungsprozess und erhöht die kognitive Belastung [40].

Optimierung des Renderings – Canvas
Ein wesentliches Merkmal der Unity-Engine ist der Mechanismus zum Rendern der Benutzeroberfläche über die Basiskomponente Canvas. Diese Komponente ist für die Generierung und Aktualisierung der Geometrie aller verschachtelten Elemente wie Schaltflächen, Textfelder und Icons verantwortlich. Die Besonderheit der Arbeitsweise des Canvas besteht

darin, dass bei der Änderung von Parametern auch nur eines einzigen untergeordneten Objekts – sei es die Bewegung eines Datenindikators, die Änderung der Transparenz einer Schaltfläche oder die Aktualisierung eines Textwerts des aktuellen Takts – ein Prozess der vollständigen Neuzeichnung des gesamten Canvas ausgelöst wird.

In der Umgebung der rechenintensiven Benutzeroberfläche des Lernspiels, in der gleichzeitig Dutzende aktiver Prozessorsignale angezeigt werden, erzeugt die Neuberechnung der gesamten Objekthierarchie in jedem Frame eine kritische Last für den CPU. Auf mobilen Endgeräten führt dies zu spürbaren Mikroruckern (Lag Spikes), die den Flow-Zustand unterbrechen und die flüssige Wahrnehmung des Lernmaterials beeinträchtigen. Bei Erreichen einer kritischen Anzahl von Objekten auf einem einzigen Canvas kann die Spielszene aufgrund des hohen Rechenaufwands für die Aktualisierung des Grafikgitters der Benutzeroberfläche faktisch unspielbar werden.

In dem in Abbildung 6.4 dargestellten Szenario sinkt die Bildrate um mehr als 15 Frames pro Sekunde (FPS), da die verfügbare Rechenleistung primär für die Aktualisierung der Geometriedaten der Benutzeroberfläche beansprucht wird, anstatt für die flüssige Darstellung der Simulationslogik.

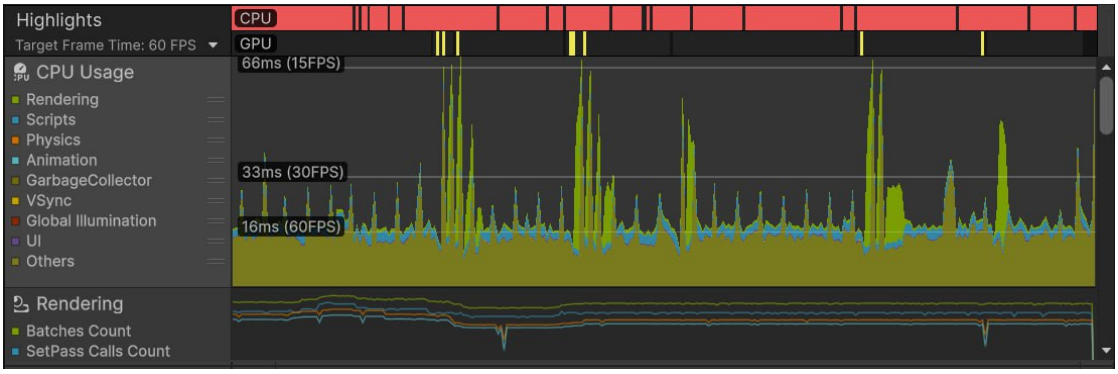


Abbildung 6.4: Signifikante Framerate-Einbrüche infolge kontinuierlicher Canvas-Rebuilds.

Zur Lösung dieses Problems wurde die Strategie der Aufteilung der Benutzeroberfläche in Sub-Canvases angewandt. Der Grundgedanke dieser Methode besteht darin, Elemente basierend auf ihrer Aktualisierungshäufigkeit voneinander zu isolieren. Dies ermöglicht es der Unity-Engine, den Canvas Rebuild-Prozess auf den jeweiligen Sub-Canvas zu lokalisieren, in dem die Änderung aufgetreten ist, ohne den Rest der Benutzeroberfläche

zu beeinflussen. Im Rahmen des Projekts wurde der übergeordnete Canvas in vier logische Ebenen unterteilt:

Hochfrequenz-Ebene
Bestimmt für dynamische Elemente der Hauptbenutzeroberfläche, wie Taktzähler und Indikatoren aktiver Datenbusse, deren Zustand sich sekündlich ändert.

Mittelfrequenz-Ebene
Beinhaltet Elemente, die auf Benutzerinteraktionen reagieren, wie Steuerungsschaltflächen oder Tooltips.

Statische Ebene
Enthält unveränderliche Gestaltungselemente, Überschriften und Hintergrundgrafiken, die nach dem initialen Laden keine Neuzeichnung erfordern.

Funktionale Blöcke
Dedizierte Canvases für unabhängige Systeme, wie das Dialogsystem oder das Konfigurationspanel des Mehrtaktprozessors. Dies erlaubt es, deren Einfluss auf die Performance der Hauptszene während der Simulation vollständig zu eliminieren.

Die Implementierung der Sub-Canvas-Hierarchie ermöglichte eine optimierte Verteilung der Geräteleistung, was eine sofortige Systemreaktion und einen stabilen Betrieb der Prozessorsimulation selbst bei maximaler visueller Belastung sicherstellte. Die Aufteilung der Benutzeroberfläche folgt dabei einer didaktischen Logik: Elemente, die sich häufig ändern — Taktzähler, Signalzustände — beanspruchen die Aufmerksamkeit des Lernenden aktiv. Elemente, die sich selten ändern — Hintergründe, Überschriften — sollten im Hintergrund verschwinden. Eine UI-Architektur, die diese Trennung nicht berücksichtigt und bei jeder Signaländerung den gesamten Canvas neu berechnet, behandelt didaktisch irrelevante Elemente wie relevante — und erzeugt damit unnötige visuelle Unruhe.

Optimierung der Texturen und Eingabeereignisse
Die abschließende Phase der grafischen UI-Optimierung umfasste die Arbeit mit dem Videospeicher und der Rechenlast des Hauptprozessors. Da Speicherressourcen in mobilen Systemen stark begrenzt sind, kann eine irrationale Nutzung von Texturen zu Anwendungsabstürzen oder unscharfen Grafiken aufgrund von Cache-Mangel führen.

Um dieses Problem zu lösen, wurden für alle Sprites und UI-Elemente moderne, für mobile Plattformen adaptierte Kompressionsalgorithmen angewandt — ASTC (Adaptive Scalable Texture Compression) [33]. Im Gegensatz zu Standard-Kompressionsmethoden ermöglicht ASTC eine flexible Anpassung des Verhältnisses zwischen Qualität und Dateigröße, was eine ideale Schärfe von Schriftarten und Icons bei minimalem Speicherverbrauch

garantiert. Zudem wurde beim Import der Grafiken der Sprite-Typ Full Rect gewählt, um die Generierung des Mesh-Gitters zu vereinfachen, und die Erstellung von Detailstufen deaktiviert, da diese in einer zweidimensionalen Benutzeroberfläche lediglich das Gewicht der Assets erhöhen, ohne einen visuellen Nutzen zu bieten.

Parallel zur Speicheroptimierung wurde die Prozessorlast durch ein Audit des Raycasting-Systems reduziert. In Unity erzwingt jedes UI-Element mit aktiviertem Raycast Target-Parameter eine frame-weise Prüfung der Kollision zwischen Cursor oder Fingertipp und dem jeweiligen Objekt. Bei einer gesättigten Benutzeroberfläche mit Dutzenden von Textfeldern und dekorativen Elementen erzeugen diese Prüfungen eine erhebliche Last für die Komponente Graphics Raycaster. Für alle nicht-interaktiven Elemente (Dekorationen, Hintergründe, beschreibende Texte) wurde dieser Parameter explizit deaktiviert. Dies ermöglichte es dem Prozessor, statische Objekte bei der Berechnung der Klick-Physik zu ignorieren, was entscheidend für die Aufrechterhaltung stabiler FPS bei aktiver Interaktion des Benutzers mit den Steuerelementen des Mehrtaktprozessors ist.

Textsysteme
Besonderes Augenmerk wurde im Projekt auf die Lesbarkeit von Textinformationen gelegt, da Lerninhalte eine hohe Zeichenscharfe auf Bildschirmen mit unterschiedlichen Auflösungen erfordern. Anstelle der Standardkomponente Unity Text, die eine veraltete Raster-Rendering-Methode verwendet, wurde das System TextMeshPro implementiert.

Der entscheidende Vorteil dieses Systems liegt in der Verwendung der SDF-Technologie (Signed Distance Field). Im Gegensatz zu Raster-Schriftarten, die beim Skalieren verpixeln, werden SDF-Schriften auf Basis mathematischer Distanzvektoren gerendert. Dies bietet dem Projekt ei Vorteile.

Zunächst lässt sich eine konsistente visuelle Qualität hervorheben, da Texte bei beliebiger Zoomstufe oder Skalierung absolut scharf bleiben. Dies ist essenziell für das Lesen kleiner Beschriftungen auf den Prozessorschemata auf mobilen Displays. Ein weiterer Vorteil ist die Speichereffizienz - anstatt mehrere Schriftgrößen im Speicher vorzuhalten, nutzt das System einen einzigen kompakten Texturatlas, aus dem die GPU dynamisch Zeichen in der benötigten Größe generiert. Zuletzt bringt dies eine bessere Performance, da das Rendering von SDF-Schriften GPU-seitig über spezialisierte Shader erfolgt, was deutlich effizienter ist als die CPU-seitige Schriftberechnung der Standardmethode.

Im didaktischen Kontext ist die visuelle Qualität der Textdarstellung unmittelbar relevant: Unscharfe oder schlecht lesbare Beschriftungen würden die kognitive Belastung der Lernenden unnötig erhöhen und die Verständlichkeit der Lerninhalte auf mobilen Displays beeinträchtigen.

6.2 Rechenoptimierung

Während die grafische Optimierung die Last auf den GPU reduziert, zielt die Rechenoptimierung auf die rationelle Nutzung der Ressourcen des CPU ab. Für mobile Endgeräte ist dies von kritischer Bedeutung, da ineffiziente Algorithmen zu einer schnellen Überhitzung des Geräts und einer daraus resultierenden Leistungsdegradation führen. Die zentrale Optimierungsstrategie dieses Projekts bestand im Übergang von einer ressourcenintensiven, permanenten Zustandsüberwachung hin zu einer ereignisgesteuerten Architektur.

6.2.1 Event-Driven Design

Ein grundlegendes Problem vieler Unity-Anwendungen ist die übermäßige Nutzung der Standardmethode `Update`, die pro Frame (bis zu 60–120 Mal pro Sekunde) aufgerufen wird. Eine permanente Abfrage von Bedingungen innerhalb dieser Methode für jeden Knoten des Mehrtaktprozessors würde eine enorme und ungerechtfertigte CPU-Last erzeugen. Da die Prozessorlogik diskret arbeitet und an ein Taktsignal gebunden ist, wäre die Verwendung von Frame-basierten Prüfungen architektonisch inkorrekt.

Zur Lösung dieser Aufgabe wurde im Projekt das Entwurfsmuster Observer realisiert. Sobald der Benutzer ein Signal generiert oder ein Taktwechsel erfolgt, wird ein entsprechendes Ereignis ausgelöst. Die auf dieses Ereignis abonnierten Komponenten werden aktiviert, führen ihre Logik aus und kehren anschließend in den Wartezustand zurück. Diese ereignisgesteuerte Architektur ermöglichte es, Leerlaufzyklen des Prozessors nahezu vollständig zu eliminieren. Dies schont nicht nur den Akku des mobilen Endgeräts, sondern sorgt auch für eine hohe Code-Qualität durch Entkopplung - die Unabhängigkeit der Objekte voneinander, was die Skalierbarkeit des Projekts bei der Implementierung neuer Prozessorbefehle erheblich vereinfacht.

6.2.2 Optimierung des Objektlebenszyklus durch Coroutines

Neben dem ereignisgesteuerten Modell war die Verwaltung langandauernder Prozesse, wie etwa die Visualisierung von Datentransfers zwischen Registern, ein wichtiger Aspekt der Optimierung. Die Verwendung von Standardschleifen innerhalb des Haupt-Threads der Anwendung könnte zu einer Blockierung der Benutzeroberfläche und zum Verlust der

Animationsflüssigkeit führen. Um die Rechenlast zeitlich zu verteilen, wurden anstelle der `Update`-Methode konsequent Coroutines eingesetzt.

Eine Koroutine ist eine spezielle Methode, die ihre Ausführung pausieren kann — entweder für eine bestimmte Zeit oder bis eine Bedingung erfüllt ist - und dabei die Kontrolle an den Hauptzyklus der Engine zurückgibt. Anstatt sekündlich zu prüfen, ob eine Animation oder Berechnung abgeschlossen ist, verbleibt das System im Ruhezustand und wird exakt zum benötigten Zeitpunkt reaktiviert. Dieser Ansatz garantiert, dass CPU-Spitzenlasten gleichmäßig über die Frames verteilt werden, was Mikroruckler verhindert. In Kombination mit dem ereignisgesteuerten Modell gewährleisteten Coroutines einen stabilen und vorhersehbaren Betrieb der Logik des Mehrtaktprozessors, unabhängig von der Leistungsfähigkeit des mobilen Endgeräts.

6.2.3 Effiziente Datenverwaltung: Scriptable Objects

Eine der zentralen architektonischen Entscheidungen des Projekts war der Übergang zum Data-Driven Design unter Verwendung der Scriptable Objects-Technologie. In mobilen Anwendungen ist es von entscheidender Bedeutung, die Spiellogik strikt von den Inhalten zu trennen, um den Verbrauch des Arbeitsspeichers zu minimieren und die Projektstruktur zu vereinfachen. Im Rahmen dieser Arbeit wurden Scriptable Objects zur Implementierung von drei fundamentalen Systemen eingesetzt.

Levelkonfigurationssystem
Anstatt neun separate, ressourcenintensive Szenen für die Endphase des Spiels (die Simulation des vollständigen Mehrtaktprozessors) zu erstellen, wurde eine Architektur nach dem Prinzip „eine Szene – mehrere Datensätze“ implementiert. Alle spezifischen Informationen für jedes der neun Level sind in separaten SO-Objekten gekapselt. Beim Start eines Levels lädt das System die benötigten Parameter dynamisch in eine einzige Basisszene. Dies ermöglichte eine radikale Reduzierung der Gesamtgröße der Anwendung und verhinderte die redundante Duplizierung von Grafikressourcen im Speicher.

Metadaten und Zustände
Die Verwendung von SO zur Speicherung kleinerer Datenmengen, wie etwa den Einstellungen für Charakteremotionen, ermöglichte eine zentralisierte Steuerung des visuellen Stils.

Graphbasiertes Dialogsystem
Die pädagogische Komponente des Spiels stützt sich auf ein eigenentwickeltes Dialogsystem, das auf einem Knotengraphen basiert. Jeder Dialogknoten sowie der Graph selbst

sind als Scriptable Objects realisiert. Diese Struktur erlaubt es, komplexe logische Verzweigungen und Metadaten (z. B. den emotionalen Zustand eines Charakters für visuelles Feedback) in Form von leichtgewichtigen Asset-Referenzen zu speichern. Dies eliminiert die Notwendigkeit, umfangreiche externe Dateien während der Laufzeit zu parsen, und gewährleistet einen verzögerungsfreien Übergang zwischen den Dialogzeilen.

Aus technischer Sicht gewährleistet diese Datenorganisation einen optimalen Speicherbedarf. Da SO-Objekte nur einmal und per Referenz in den Speicher geladen werden, wird eine Informationsduplizierung ausgeschlossen. Dies garantiert einen stabilen Betrieb der Anwendung auch auf Geräten mit begrenztem Arbeitsspeicher.

6.2.4 Animationsoptimierung mit DOTween

Zur Schaffung einer hochwertigen Benutzererfahrung und einer hohen Schnittstellenflüssigkeit wurde im Projekt die Programm-Animationsbibliothek DOTween eingesetzt. In der mobilen Entwicklung ist die Standardkomponente Unity Animator für UI-Aufgaben oft redundant, da sie permanente Zustandsberechnungen erfordert und selbst im Leerlauf signifikante CPU-Ressourcen beansprucht.

Der Einsatz von DOTween ermöglichte es, schwerfällige Animationscontroller durch leichtgewichtige mathematische Berechnungen (Interpolation) auf Code-Ebene zu ersetzen.

Diese Lösung bietet primär den Vorteil einer Minimierung der CPU-Last, da sogenannte „Tweens“ im Gegensatz zum Animator nur zum Zeitpunkt der eigentlichen Aktion Ressourcen verbrauchen. Die programmgesteuerte Berechnung von Trajektorien und Transparenzwerten der UI-Elemente erfolgt über optimierte Algorithmen, was für die Aufrechterhaltung stabiler Bildraten (FPS) auf leistungsschwächeren Geräten kritisch ist.

Neben der technischen Performance trägt dieser Ansatz zur Verbesserung der kognitiven Wahrnehmung bei. Flüssige Übergänge und die dynamische Reaktion der Benutzeroberfläche auf Benutzeraktionen — im Gamedesign oft als „Juiciness“ bezeichnet — sind dabei nicht nur ästhetische Entscheidungen, sondern haben eine direkte kognitive Funktion. Nach der CLT (siehe Kapitel 2.2.3) unterstützen konsistente visuelle Rückmeldungen den Aufbau mentaler Schemata: Der Lernende verknüpft die Animation des Signalbusses mit dem Vorgang im Datenpfad. Eine ruckelhafte oder verzögerte Animation würde diese Verknüpfung stören — der Lernende sähe nicht mehr Ursache und Wirkung, sondern zwei getrennte Ereignisse. In diesem Sinne ist die Animationsflüssigkeit kein Komfort, sondern eine Voraussetzung für das korrekte Verstehen der Prozessorlogik.

Schließlich lässt sich sagen, dass kein Studierender jemals bemerken wird, dass Lightmaps statt Echtzeit-Schatten verwendet werden oder dass die UI in Sub-Canvases aufgeteilt ist. Das ist der Punkt. Gute technische Optimierung ist unsichtbar — sie fällt nur auf, wenn sie fehlt.

7 Evaluation und Testing

7.1 Methodik und Durchführung

Die Evaluierung des Projekts gliedert sich in zwei wesentliche Phasen, um sowohl die Anforderungen der Zielgruppe als auch die Effektivität der entwickelten Lösung zu untersuchen. Die erste Phase bestand aus einer initialen Bedarfsanalyse zu Beginn des Projekts, während die zweite Phase eine umfassende Nutzerstudie mit einem funktionalen Prototyp (Version 0.6.8) gegen Ende des Semesters umfasste. Obwohl zum Zeitpunkt der Fertigstellung dieser Arbeit bereits die Version 0.7.6 vorliegt, liefert die Evaluierung der Vorgängerversion repräsentative Daten zur Usability und zum didaktischen Mehrwert.

7.1.1 Methodik und Testaufbau

Die erste Erhebung wurde als quantitative Online-Umfrage konzipiert. Ziel dieser Bedarfsanalyse war es, die spezifischen Herausforderungen der Studierenden im Modul „Rechnerorganisation“ zu identifizieren und die Akzeptanz für ein digitales Lernspiel zu validieren. Der Fragebogen umfasste 14 geschlossene Fragen zur Selbsteinschätzung der Lernschwierigkeiten sowie zwei offene Fragen für qualitatives Feedback. Insgesamt nahmen 45 Studierende der Informatik an dieser Vorstudie teil.

Die zweite Evaluationsphase fokussierte sich auf die praktische Erprobung des Lernspiels. Die Stichprobe von 12 Probanden setzte sich aus Studierenden verschiedener Fachbereiche der TU Darmstadt zusammen. Trotz der begrenzten Teilnehmerzahl wurde auf eine heterogene Zusammensetzung geachtet: Die Gruppe war paritätisch nach Geschlecht (50% weiblich, 50% männlich) sowie nach Vorwissen aufgeteilt. Genau die Hälfte der Teilnehmenden hatte das Modul „Rechnerorganisation“ bereits erfolgreich absolviert, während die andere Hälfte über keinerlei Vorkenntnisse in der Prozessorarchitektur verfügte.

Diese Aufteilung ermöglichte es, die Lernkurve für unterschiedliche Ausgangsniveaus zu simulieren und zu vergleichen.

Als zentrales Messinstrument für die Nutzerzufriedenheit wurde die System Usability Scale (SUS) nach Brooke eingesetzt [6]. Der SUS ist ein standardisierter Fragebogen aus zehn Items, dessen Gesamtwert auf einer Skala von 0 bis 100 Punkten berechnet wird. Werte ab 85 gelten als „Excellent“, Werte zwischen 71 und 85 als „Good“ — eine Einordnung, die sich in der Forschungsgemeinschaft als Industriestandard etabliert hat [3]. Die Entscheidung für den SUS fiel aufgrund seiner hohen Reliabilität und der Vergleichbarkeit der Ergebnisse mit Industriestandards. Da eigene Entwürfe für Fragebögen oft nicht die methodische Validität etablierter Skalen erreichen, wurde der SUS lediglich um spezifische Fragen zum Lernerfolg und zur individuellen Nutzererfahrung ergänzt. Einschränkend ist anzumerken, dass die Stichprobengröße von 12 Probanden keine statistisch repräsentativen Aussagen erlaubt. Die vorliegende Studie ist daher als explorative Pilotstudie zu verstehen, deren Ergebnisse als erste Orientierungswerte dienen und in einer größer angelegten Folgestudie zu validieren wären. Zudem ist die physische Präsenz des Entwicklers während der Testdurchführung als potenzielle Quelle sozialer Erwünschtheit zu berücksichtigen.

7.1.2 Durchführung der Nutzertests

Erste Testphase (Bedarfsanalyse)
Die Distribution erfolgte über fachspezifische Kommunikationskanäle der Informatik-Studierendenschaft. Da die Umfrage anonym durchgeführt wurde, basieren die Ergebnisse auf der authentischen Selbstauskunft der Teilnehmenden bezüglich ihrer Zugehörigkeit zur Zielgruppe. Die hohe Rücklaufquote innerhalb kurzer Zeit unterstreicht das Interesse der Studierenden an alternativen Lehrmethoden.

Zweite Testphase (Prototyp-Testing)
Die Nutzertests wurden als kontrollierte Einzelsitzungen durchgeführt. Den Probanden stand es frei, das Spiel auf ihrem eigenen Android-Endgerät (Bring Your Own Device), einem bereitgestellten Laptop (Windows) oder einem vorkonfigurierten mobilen Testgerät zu explorieren. Da die mobile Optimierung im Fokus dieser Arbeit steht, beziehen sich die evaluierten Daten primär auf die mobile Nutzungserfahrung.

Der Testablauf war wie folgt strukturiert: Die Teilnehmenden absolvierten eigenständig die Lerninhalte bis zum Kapitel „Extender“ (5. Kapitel von 7). Die verbleibenden fortgeschrittenen Kapitel wurden anschließend demonstriert und standen für eine explorative

Erprobung ohne explizite Lösungsanforderung zur Verfügung. Eine durchschnittliche Testsession dauerte inklusive der anschließenden Befragung etwa 40 Minuten. Den Abschluss bildete ein kurzes, unstrukturiertes Interview (Rückmeldung), um subjektive Eindrücke zu erfassen, die über die standardisierten Fragen hinausgingen. Auf eine systematische Dokumentation dieser Rückmeldungen wurde bewusst verzichtet, da im Rahmen dieser Studie der Fokus auf quantitativ auswertbaren Daten lag. Die qualitativen Eindrücke flossen jedoch informell in die Weiterentwicklung des Prototyps ein.

7.2 Analyse

In diesem Kapitel werden die Ergebnisse der beiden durchgeführten Erhebungen präsentiert und evaluiert. Die Analyse folgt der chronologischen und logischen Abfolge des Projekts: Zunächst wird die initiale Bedarfsanalyse ausgewertet, um die Problemfelder der Studierenden zu präzisieren. Anschließend folgt die Analyse des Nutzertests mit dem funktionalen Prototypen.

7.2.1 Quantitative Auswertung der Bedarfsanalyse

An der ersten Umfrage nahmen insgesamt 45 Studierende teil. Die Stichprobe setzt sich primär aus Studierenden der Informatik (95,6%, 43 Personen) zusammen, ergänzt durch einzelne Teilnehmende aus den Studiengängen „Informatik Lehramt an Gymnasien“ und „Informationssystemtechnik“. Bezüglich des Studienfortschritts gaben 71,1% (32 Personen) an, das Modul Rechnerorganisation bereits bestanden zu haben. 24,4% (11 Personen) bereiten sich auf eine Wiederholungsprüfung vor, während 4,4% das Modul abgebrochen haben.

Wahrnehmung der Modulqualität und Übungsbetrieb
Die allgemeine Schwierigkeit des Moduls wird von den Teilnehmenden im Median als „ausgewogen“ bewertet. Dies deutet darauf hin, dass die Übungsmaterialien und die Prüfungsanforderungen grundsätzlich auf das Niveau der Studierenden abgestimmt sind. Während die Qualität der Vorlesungsinhalte überwiegend positiv bewertet wurde (45% „gut“ oder „sehr gut“ gegenüber 17% „schlecht“ oder „sehr schlecht“), zeigt sich beim Übungsbetrieb ein differenzierteres Bild. Lediglich 4,5% der Befragten bewerteten die Übungen eindeutig als „gut“.

Diese Diskrepanz lässt sich potenziell auf die heterogene Qualität der tutorziellen Betreuung zurückführen. Aufgrund des Professorenwechsels im Sommersemester 2025 (SoSe25) und der damit verbundenen Umstellung des Curriculums auf die RISC-V-Architektur sowie neue Themen wie Cache-Hierarchien, standen die Tutoren vor der Herausforderung, neue Inhalte in kurzer Zeit zu adaptieren. Qualitative Beobachtungen und Gespräche im akademischen Umfeld (WiSe22 bis WiSe25) lassen vermuten, dass die Auswahlkriterien für Tutoren — oft primär basierend auf der eigenen Modulnote — nicht immer die notwendige didaktische Tiefe oder die Anpassungsfähigkeit an neue Lehrinhalte garantieren. Dies führt zu einer Inkonsistenz im Übungsbetrieb, durch die die hohe Qualität der Vorlesungsmaterialien in der praktischen Vermittlung teilweise verloren geht.

Identifikation komplexer Themenbereiche
Als besonders herausfordernd wurden die Themen Pipeline- und Mehrtaktprozessor sowie die Assemblerprogrammierung identifiziert. Diese Ergebnisse decken sich mit den Erfahrungen der Fachgruppe ESA an der TU Darmstadt. In Abstimmungsgesprächen im Rahmen dieser Arbeit wurde bestätigt, dass Studierende insbesondere Schwierigkeiten haben, die Dynamik des Datenpfads zu verstehen und Assembler-Code für komplexe Hardware-Strukturen eigenständig zu entwerfen.

Diese Einschätzung spiegelt sich auch in der Analyse der Klausuraufgaben wider: Die Aufgabe zum Mehrtaktprozessor wurde von 26,7% der Befragten als die schwierigste empfunden, gefolgt vom Pipelineprozessor und Assembler-Aufgaben (jeweils 17,8%). Kritisch zu betrachten ist, dass 37,8% der Studierenden angaben, die Übungen und Testate hätten nur unzureichend (28,9%) oder gar nicht (8,9%) zur Prüfungsvorbereitung beigetragen. Dies legt nahe, dass der aktuelle Übungsmodus die notwendige Transferleistung für diese komplexen Themen nur bedingt fördert.

Potenzial eines Lernspiels (Game-Based Learning)
Die Bereitschaft der Studierenden, digitale Lernspiele zur Vertiefung der Inhalte zu nutzen, ist sehr hoch: 53,3% antworteten mit „Ja“, weitere 42,2% mit „Eher Ja“. Als bevorzugtes Genre wurde mit 45,5% die Simulation (z. B. der Bau eines virtuellen Rechners) gewählt. Die drei wichtigsten Anforderungen an ein solches Tool sind aus Sicht der Lernenden hohe Verständlichkeit der Aufgaben (80%), direktes Feedback bei Fehlern (66,7%) und Spaß und Motivation (46,7%). Interessanterweise gaben 62,2% der Befragten an, bereit zu sein, 1 bis 3 Stunden pro Woche in ein solches Lernspiel zu investieren. Dies entspricht zeitlich dem Umfang eines regulären Übungsbesuchs, bietet jedoch eine deutlich höhere zeitliche und örtliche Flexibilität. Im Gegensatz zum wöchentlichen Übungstermin ermöglicht eine App den Studierenden, Fragen und Verständnisprobleme unmittelbar dann zu bearbeiten, wenn sie auftreten, ohne auf den nächsten Termin oder die Verfügbarkeit eines Tutors

angewiesen zu sein. Das hohe Interesse lässt darauf schließen, dass Studierende nicht primär nach Unterhaltung suchen, sondern nach einem effizienten, interaktiven Werkzeug, das die methodischen Lücken des traditionellen Übungsbetriebs schließt.

7.2.2 Quantitative Auswertung der Pilotierung

Nach der Analyse der Bedarfsanalyse liegt der Fokus im Folgenden auf der Evaluierung des funktionalen Prototyps (Version 0.6.8). Die Ergebnisse der Pilotierung mit 12 Probanden offenbarten ein differenziertes Bild hinsichtlich des Einstiegs in die Spielmechanik. Interessanterweise zeigten sich bei den ersten Leveln widersprüchliche Bewertungen durch die heterogene Probandengruppe. Während ein Teil der Teilnehmenden die Einstiegshürden als sehr niedrig empfand, bewerteten andere die initialen Aufgaben als moderat fordernd. Eine Korrelation zwischen dem fachlichen Vorwissen und der subjektiv wahrgenommenen Schwierigkeit ließ sich dabei nicht eindeutig feststellen; so gab es Probanden ohne Vorkenntnisse, die die Spielmechanik intuitiv adaptierten, während vereinzelt erfahrene Studierende Schwierigkeiten mit der Benutzeroberfläche und der Zielsetzung der Level äußerten.

Auffallend war jedoch, dass sich diese Diskrepanzen zur Mitte des Spiels hin auflösten. Ab diesem Zeitpunkt wurde die Struktur und Komplexität der Level durchweg positiv und konsistent bewertet. Da der Großteil der Testpersonen angab, nur selten oder gar nicht digital zu spielen, kann eine Verzerrung der Ergebnisse durch übermäßige Gaming-Erfahrung (Expert Bias) ausgeschlossen werden. Die Aufteilung der Lerninhalte in Kapitel sowie die allgemeine Spielstruktur wurden als lernförderlich wahrgenommen, wobei die Probanden nach der Überwindung der initialen Lernkurve der Steuerung weniger Aufmerksamkeit schenken mussten und sich verstärkt auf die Logik des Mehrtaktprozessors konzentrieren konnten.

Ein wesentlicher Aspekt der Untersuchung war das Erleben des sogenannten Flow-Zustands. Die Beobachtungen während der Testphasen deuteten darauf hin, dass die Anwendung bei einem Großteil der Teilnehmenden eine hohe Motivation auslöste.

Zusätzlich zur funktionalen Ebene wurde die ästhetische Gestaltung des Prototyps hervorgehoben. Die Hälfte der Teilnehmenden lobte explizit die Grafik und das Interface-Design, wobei ein Rezipient die visuelle Gestaltung sogar als „beruhigend“ bezeichnete. Dieser Kontrast zu den oft funktional reduzierten Anwendungen im akademischen Alltag oder herkömmlichen Serious Games bestätigt das Ziel des Projekts, eine ansprechende und professionelle Nutzeroberfläche zu schaffen, die die kognitive Belastung eher senkt als

erhöht. Die quantitative Auswertung der System Usability Scale (SUS) ergab einen Durchschnittswert von 80 Punkten, was die Anwendung im Bereich „Excellent“ verortet. Die detaillierte Interpretation der SUS-Daten zeigt, dass der Schwierigkeitsgrad der Level als fordernd, aber nicht frustrierend wahrgenommen wird und die theoretischen Einführungen zu Beginn der Kapitel den Informationsbedarf der Studierenden vollständig decken. Trotz einer vereinzelt geäußerten Orientierungslosigkeit zu Beginn des Spiels bewertete die Mehrheit den logischen Aufbau der Inhalte als stimmig, sodass keine externen Hilfsmittel zur Themenerschließung notwendig waren. Besonders hervorzuheben ist die hohe Nutzungsintention der Befragten. Die Validität dieser positiven Resonanz wird durch die Beobachtung gestützt, dass mehrere Probanden die Applikation über den Testzeitraum hinaus nutzten, um die Simulation vollständig abzuschließen – darunter auch Personen, die das Modul Rechnerorganisation gar nicht belegen müssen.

Die vorliegenden Daten bieten dennoch einen fundierten ersten Beleg für die Wirksamkeit und Akzeptanz des Prototyps.

8 Fazit und Ausblick

8.1 Zusammenfassung der Ergebnisse

In diesem Kapitel erfolgt eine kritische Reflexion der im ersten Kapitel 1.3 definierten Zielsetzungen im Vergleich zum realisierten Projektstand (Version 0.7.6). Die Analyse verdeutlicht, dass die angestrebten didaktischen und technischen Meilensteine erreicht wurden.

Didaktische Zielsetzung und Wissensvermittlung
Das primäre Ziel, die komplexen Abläufe eines Mehrtaktprozessors interaktiv zu vermitteln, wurde durch einen inkrementellen Lernpfad realisiert. Die Anwendung führt die Nutzenden systematisch von den basalen Hardware-Komponenten über funktionale Teilbaugruppen bis hin zur Simulation der vollständigen Prozessorarchitektur. Ein wesentlicher Aspekt des Erfolgs ist der didaktische Brückenschlag zwischen der abstrakten Theorie der Finite State Machine (FSM) und der physischen Signalverarbeitung. Indem die Spielenden die Rolle der Control Unit übernehmen, treffen sie eigenständig Entscheidungen über den Signalfluss im Datenpfad. Die unmittelbare visuelle Rückmeldung über die Konsequenzen dieser Aktionen ermöglichte die angestrebte Transferleistung: Das Mapping von Zustandsübergängen auf die tatsächliche Hardware-Logik wurde für die Probanden greifbar und verständlich. Die Evaluation in den vorangegangenen Abschnitten bestätigt, dass dieses aktive Experimentieren das Verständnis der RISC-V-Architektur maßgeblich fördert.

Eine repräsentative Aussage über den tatsächlichen Lernerfolg im Sinne eines messbaren Wissenstransfers war im Rahmen dieser Bachelorarbeit jedoch nicht möglich: Die Stichprobengröße von 12 Probanden erlaubt keine statistisch belastbaren Schlussfolgerungen. Eine groß angelegte Längsschnittstudie wäre notwendig, um die didaktische Wirksamkeit des Lernspiels wissenschaftlich fundiert zu belegen.

Technische Umsetzung und Optimierung
Auch die softwaretechnischen Zielsetzungen wurden vollständig umgesetzt. Die Anwendung wurde erfolgreich innerhalb der Unity-Engine für das Betriebssystem Android konzipiert und optimiert. Trotz der inhärent hohen Informationsdichte einer Prozessorsimulation gelang es, ein Benutzerinterface (UI) zu gestalten, das auf mobilen Endgeräten sowohl eine präzise Steuerung als auch eine klare Übersichtlichkeit gewährleistet. Die technische Stabilität der Simulations-Logik ermöglichte einen flüssigen Spielablauf, der für die Aufrechterhaltung des Flow-Zustands bei den Lernenden essenziell war.

Distribution und Professionalisierungsgrad
Als abschließender Beleg für die Qualität und die angestrebte Abgeschlossenheit des Projekts wurde die Anwendung auf den Distributionsplattformen Google Play Store und itch.io veröffentlicht. Dieser Schritt geht über den reinen Prototypenstatus hinaus und demonstriert die Marktreife sowie die technische Integrität des Produkts. Die Verfügbarkeit auf diesen Plattformen unterstreicht den Anspruch der Arbeit, eine ernsthafte und professionell aufbereitete Ergänzung zur universitären Lehre darzustellen.

Im Rahmen der gesetzten technischen und konzeptionellen Ziele dieser Bachelorarbeit wurden die definierten Anforderungen nach aktuellem Erkenntnisstand erfüllt. Die Frage nach der langfristigen didaktischen Wirksamkeit bleibt bewusst offen und stellt den natürlichen Ausgangspunkt für weiterführende Forschung dar.

8.2 Zukünftige Erweiterungsmöglichkeiten

Die nachfolgenden Erweiterungsmöglichkeiten lassen sich grob in zwei Kategorien einteilen: kurzfristig umsetzbare Maßnahmen und langfristige Entwicklungsziele, die einen deutlich größeren Aufwand erfordern.

Obwohl die aktuelle Version 0.7.6 einen abgeschlossenen Prototyp darstellt und ihre Hauptfunktion erfüllt, bestehen für dieses Projekt zahlreiche Weiterentwicklungsmöglichkeiten, die im Folgenden dargestellt werden.

Da die Anwendung gezielt auf die Bedürfnisse von Studierenden der TU Darmstadt ausgerichtet wurde, stellt die Übersetzung der Anwendung und ihrer Inhalte in weitere Sprachen einen naheliegenden nächsten Schritt dar. Derzeit ist die Anwendung ausschließlich auf Deutsch verfügbar, wodurch ein erheblicher Teil der Studierenden ausgeschlossen wird. Eine Mehrsprachigkeit könnte das Verständnis der Inhalte für internationale Studierende

deutlich verbessern. Zudem verfügt der Entwickler über Kenntnisse mehrerer Sprachen, wodurch eine eigenständige Übersetzung möglich ist.

Eine weitere sinnvolle Erweiterung besteht in der Integration zusätzlicher Themen aus dem Bereich der Rechnerorganisation, sodass Studierende das gesamte Modul mithilfe der Anwendung erlernen können. Bereits im aktuellen Entwicklungsstand ist eine Erweiterung um Inhalte wie den Eintaktprozessor und den Pipelineprozessor möglich. Da im Rahmen des Projektes der Mehrtaktprozessor ausführlich behandelt wird, bestehen bereits inhaltliche Überschneidungen mit der Digitaltechnik. Um ein vollständiges und kohärentes Lernangebot zu schaffen, könnten daher auch alle relevanten Themen der Digitaltechnik integriert werden. Dies würde eine durchgängige Lernumgebung ermöglichen – von grundlegenden Konzepten bis hin zu komplexeren Zusammenhängen der Hardware – unterstützt durch anschauliche Beispiele und Simulationen.

Damit eine derart umfangreiche Lernanwendung nicht zu einer bloßen Sammlung interaktiver Aufgaben degeneriert, bietet sich die Entwicklung eines narrativen Rahmens an. Eine durchdachte Spielwelt, eine konsistente Handlung sowie klar definierte Ziele könnten die Motivation der Nutzenden steigern. Durch die Einbettung der Lerninhalte in eine Geschichte würde der Fokus von reinem Auswendiglernen auf das Erreichen spielinterner Ziele verlagert, was sich positiv auf die Lernmotivation und das Engagement auswirken kann.

Wie bereits erwähnt, setzt das Spiel aktuell ein gewisses Vorwissen voraus, insbesondere im Kontext der TU Darmstadt. Begriffe wie RISC-V-Architektur, konkrete Prozessorarchitekturen oder die Module Rechnerorganisation und Digitaltechnik werden teilweise implizit verwendet. Auch wenn dieses Wissen für das erfolgreiche Absolvieren des Spiels nicht zwingend erforderlich ist, kann es bei unerfahrenen Nutzenden zu Verständnisproblemen führen. Eine mögliche Erweiterung besteht daher darin, das Spiel so zu gestalten, dass es auch für Personen ohne Vorkenntnisse zugänglich ist. Dies könnte durch zusätzliche Erklärungen erfolgen, die über den Lehrplan hinausgehen, beispielsweise durch Vergleiche zwischen RISC-V- und ARM-Architekturen oder durch praxisnahe Anwendungsbeispiele.

In Kapitel 7.2.2 wurde festgestellt, dass etwa die Hälfte der Befragten das Design und den Stil des Spiels positiv bewertet hat. Da derzeit visuelle Elemente aus frei verfügbaren Ressourcen verwendet werden, stellt die Entwicklung eines individuellen Designs einen weiteren wichtigen Verbesserungsschritt dar. Ein maßgeschneidertes visuelles Konzept könnte das Spiel stärker von anderen Anwendungen abheben und gleichzeitig die narrative Ebene unterstützen. Neben der visuellen Gestaltung spielt auch die akustische Komponente eine wesentliche Rolle. Aktuell verfügt das Spiel weder über Musik noch über Soundeffekte. Die Integration entsprechender Audioelemente würde das Feedback für die Nutzenden

verbessern und die Immersion erhöhen. Aufgrund der Komplexität des Audio-Designs sollte dieser Aspekt jedoch als eigenständiger Entwicklungsschritt betrachtet werden.

Gamification-Elemente wie Badges haben sich in Lernumgebungen als motivierend erwiesen. Daher stellt deren Integration eine sinnvolle Erweiterung dar. Im Kontext einer plattformübergreifenden Anwendung könnten sowohl lokale als auch globale Achievements implementiert werden. Plattformen wie Google Play oder Steam unterstützen entsprechende Funktionen, die relativ einfach eingebunden werden können und die Motivation der Nutzenden zusätzlich steigern.

Obwohl die Anwendung primär für mobile Geräte mit einer Portierung auf PC konzipiert wurde, besteht langfristig auch die Möglichkeit, sie auf Spielkonsolen auszuweiten, um eine breitere Zielgruppe zu erreichen.

Nach der Umsetzung der genannten Erweiterungen könnte zudem ein lokaler sowie plattformübergreifender kooperativer Modus integriert werden. Dies würde es ermöglichen, das Spiel gemeinsam mit anderen Studierenden oder Lehrenden zu nutzen und Lerninhalte kollaborativ zu erarbeiten.

Schließlich besteht die Möglichkeit, das Anwendungssystem durch pädagogische Zertifizierungen weiter aufzuwerten. Viele Distributionsplattformen bieten entsprechende Programme an, wie beispielsweise die Funktion „Teacher Approved“ im Google Play Store. Alternativ kann auch eine unabhängige Zertifizierung angestrebt werden, um die Qualität und den Bildungswert der Anwendung hervorzuheben.

Literatur

[1] Gabriel Almir. *PhotoGIMP*. 2025. URL: https://github.com/Diolinux/PhotoGIMP.git.

[2] Paul Ayres und John Sweller. „The split-attention principle in multimedia learning“. In: *The Cambridge handbook of multimedia learning* 2 (2005), S. 135–146.

[3] Aaron Bangor, Philip Kortum und James Miller. „Determining what individual SUS scores mean: Adding an adjective rating scale“. In: *Journal of usability studies* 4.3 (2009), S. 114–123.

[4] Martin Boeker u. a. „Game-Based E-Learning Is More Effective than a Conventional Instructional Method: A Randomized Controlled Trial with Third-Year Medical Students“. In: *PLOS ONE* 8.12 (Dez. 2013), null. DOI: 10.1371/journal.pone.0082328. URL: https://doi.org/10.1371/journal.pone.0082328.

[5] Sarah Bordac und Jean Rainwater. „User-Centered Design in Practice: The Brown University Experience“. In: *Journal of Web Librarianship* 2.2-3 (2008), S. 109–138. DOI: 10.1080/19322900802205908. eprint: https://doi.org/10.1080/19322900802205908. URL: https://doi.org/10.1080/19322900802205908.

[6] John Brooke u. a. „SUS-A quick and dirty usability scale“. In: *Usability evaluation in industry* 189.194 (1996), S. 4–7.

[7] Michelene T. H. Chi und Rod D. Roscoe. „The Processes and Challenges of Conceptual Change“. In: *Reconsidering Conceptual Change: Issues in Theory and Practice*. Hrsg. von Margarita Limón und Lucia Mason. Dordrecht: Springer Netherlands, 2002, S. 3–27. ISBN: 978-0-306-47637-2. DOI: 10.1007/0-306-47637-1_1. URL: https://doi.org/10.1007/0-306-47637-1_1.

[8] Tomorrow Corporation. *Human Resource Machine*. Google Play. 2016. URL: https://play.google.com/store/apps/details?id=com.tomorrowcorporation.humanresourcemachine&hl=de.

[9] Mihaly Csikszentmihalyi. *Flow: The psychology of happiness*. Random House, 2013.

[10] André Czauderna und Emmanuel Guardiola. „The gameplay loop methodology as a tool for educational game design“. In: *Electronic Journal of e-Learning* 17.3 (2019), pp207–221.

[11] Sebastian Deterding u. a. „From game design elements to gamefulness: defining"gamification"“. In: *Proceedings of the 15th international academic MindTrek conference: Envisioning future media environments*. 2011, S. 9–15.

[12] ephtracy. *MagicaVoxel*. 2026. URL: https://ephtracy.github.io/index.html?page=mv_main.

[13] Kristin L Fraser, Paul Ayres und John Sweller. „Cognitive load theory for the design of medical simulations“. In: *Simulation in Healthcare* 10.5 (2015), S. 295–307.

[14] Markus Fuchs und Christian Wolff. „Improving programming education through gameful, formative feedback“. In: *2016 ieee global engineering education conference (educon)*. IEEE. 2016, S. 860–867.

[15] Funghisoft. *MHRD*. Steam Store. 2017. URL: https://store.steampowered.com/app/576030/MHRD/?l=german.

[16] game – Verband der deutschen Games-Branche e.V. *Marktdaten: Zahlen & Fakten*. 2024. URL: https://www.game.de/zahlen-und-fakten/.

[17] PleasingFungus Games. *Silicon Zeroes*. Steam Store. 2017. URL: https://store.steampowered.com/app/684270/Silicon_Zeroes/.

[18] Daniele Giardini. *DOTween*. 2026. URL: https://dotween.demigiant.com/index.php.

[19] Joshua Gold und Joseph Ciorciari. „A Review on the Role of the Neuroscience of Flow States in the Modern World“. In: *Behavioral Sciences* 10.9 (2020). ISSN: 2076-328X. DOI: 10.3390/bs10090137. URL: https://www.mdpi.com/2076-328X/10/9/137.

[20] Sarah Harris und David Harris. *Digital Design and Computer Architecture, RISC-V Edition*. Morgan Kaufmann, 2021.

[21] John Hattie und Helen Timperley. „The power of feedback“. In: *Review of educational research* 77.1 (2007), S. 81–112.

[22] JustCreate. *Low-Poly Simple Nature Pack*. 2023. URL: https://assetstore.unity.com/packages/3d/environments/landscapes/low-poly-simple-nature-pack-162153.

[23] LevelHead. *Turing Complete*. Steam Store. 2021. URL: https://store.steampowered.com/app/1444480/Turing_Complete/.

[24] L. Li, K. F. Hew und J. Du. „Gamification enhances student intrinsic motivation, perceptions of autonomy and relatedness, but minimal impact on competency: a meta-analysis and systematic review“. In: *Educational technology research and development* 72 (2024), S. 765–796. DOI: 10.1007/s11423-023-10337-7.

[25] Marcia C Linn. „Designing computer learning environments for engineering and computer science: The scaffolded knowledge integration framework“. In: *Journal of Science Education and technology* 4.2 (1995), S. 103–126.

[26] Raymond Lister u. a. „Not seeing the forest for the trees: novice programmers and the SOLO taxonomy“. In: *ACM SIGCSE Bulletin* 38.3 (2006), S. 118–122.

[27] Daniel López-Fernández u. a. „Comparing Traditional Teaching and Game-Based Learning Using Teacher-Authored Games on Computer Science Education“. In: *IEEE Transactions on Education* 64.4 (2021), S. 367–373. DOI: 10.1109/TE.2021.3057849.

[28] F. MARTON und R. SÄLJÖ. „ON QUALITATIVE DIFFERENCES IN LEARNING: I—OUTCOME AND PROCESS“. In: *British Journal of Educational Psychology* 46.1 (1976), S. 4–11. DOI: https://doi.org/10.1111/j.2044-8279.1976.tb02980.x. eprint: https://bpspsychub.onlinelibrary.wiley.com/doi/pdf/10.1111/j.2044-8279.1976.tb02980.x. URL: https://bpspsychub.onlinelibrary.wiley.com/doi/abs/10.1111/j.2044-8279.1976.tb02980.x.

[29] Richard E Mayer und Celeste Pilegard. „Principles for managing essential processing in multimedia learning: Segmenting, pretraining, and modality principles“. In: *The Cambridge handbook of multimedia learning* 13.316-44 (2005), S. 1.

[30] Newzoo. *Games Market Year in Review 2025 to Date*. 2025. URL: https://newzoo.com/resources/blog/year-in-review-2025-to-date.

[31] Jakob Nielsen. *Usability engineering*. Morgan Kaufmann, 1994.

[32] Erik G Nilsson. „Design patterns for user interface for mobile applications“. In: *Advances in engineering software* 40.12 (2009), S. 1318–1328.

[33] Jörn Nystad u. a. „Adaptive scalable texture compression“. In: *Proceedings of the Fourth ACM SIGGRAPH/Eurographics Conference on High-Performance Graphics*. 2012, S. 105–114.

[34] Morten Borup Petersen. *Ripes*. GitHub. 2023. URL: https://github.com/mortbopet/ripes.

[35] Jan L Plass, Bruce D Homer und Charles K Kinzer. „Foundations of game-based learning“. In: *Educational psychologist* 50.4 (2015), S. 258–283.

[36] Bit Flip Productions. *Circuit Snap*. Google Play. 2023. URL: https://play.google.com/store/apps/details?id=com.bitflipproductions.circuitsnap&hl=gsw.

[37] Yizhou Qian und James Lehman. „Students' misconceptions and other difficulties in introductory programming: A literature review“. In: *ACM Transactions on Computing Education (TOCE)* 18.1 (2017), S. 1–24.

[38] Noam Nisan & Shimon Schocken. *Nand2Tetris*. Nand2Tetris Project. 2017. URL: https://www.nand2tetris.org/.

[39] Scott Steffes. *VertexTextAnimationDemo*. 2020. URL: https://github.com/markv12/VertexTextAnimationDemo.git.

[40] John Sweller. „Cognitive load during problem solving: Effects on learning“. In: *Cognitive Science* 12.2 (1988), S. 257–285. ISSN: 0364-0213. DOI: https://doi.org/10.1016/0364-0213(88)90023-7. URL: https://www.sciencedirect.com/science/article/pii/0364021388900237.

[41] The GIMP Development Team. *GIMP — GNU Image Manipulation Program*. 2024. URL: https://www.gimp.org/.

[42] Mohib Ullah u. a. „Serious games in science education: a systematic literature“. In: *Virtual Reality & Intelligent Hardware* 4.3 (2022). Advances in Wireless Sensor Networks under AI-SG forAugmented Reality Special Issue, S. 189–209. ISSN: 2096-5796. DOI: https://doi.org/10.1016/j.vrih.2022.02.001. URL: https://www.sciencedirect.com/science/article/pii/S2096579622000201.

[43] Unity Technologies. *Create modular game architecture in Unity with ScriptableObjects (Unity 6 edition)*. E-book. 2025. URL: https://unity.com/resources/create-modular-game-architecture-scriptableobjects-unity-6.

[44] Unity Technologies. *Introduction to DOTS concepts, features, & samples for advanced Unity developers (Unity 6 edition)*. E-book. 2025. URL: https://unity.com/resources/dots-concepts-features-samples-resources-unity-6.

[45] Unity Technologies. *Level up your code with design patterns and SOLID*. E-book. 2024. URL: https://unity.com/resources/design-patterns-solid-ebook.

[46] Unity Technologies. *Optimize your game performance for mobile, XR, and the web in Unity (Unity 6 edition)*. E-book. 2024. URL: https://unity.com/resources/mobile-xr-web-game-performance-optimization-unity-6.

[47] Unity Technologies. *UI Toolkit for advanced Unity developers (Unity 6 edition)*. E-book. 2025. URL: https://unity.com/resources/scalable-performant-ui-uitoolkit-unity-6.

[48] Unity Technologies. *Ultimate guide to profiling Unity games (Unity 6 edition)*. E-book. 2025. URL: https://unity.com/resources/ultimate-guide-to-profiling-unity-games-unity-6.

[49] Lev Semenovich Vygotsky und Michael Cole. *Mind in society: Development of higher psychological processes*. Harvard university press, 1978.

[50] Liang-Hui Wang u. a. „Effects of digital game-based STEM education on students' learning achievement: a meta-analysis". In: *International Journal of STEM Education* (17. März 2022). DOI: 10.1186/s40594-022-00344-0.

[51] H.L. Woo. „Designing multimedia learning environments using animated pedagogical agents: factors and issues". In: *Journal of Computer Assisted Learning* 25.3 (2009), S. 203–218. DOI: https://doi.org/10.1111/j.1365-2729.2008.00299.x. eprint: https://onlinelibrary.wiley.com/doi/pdf/10.1111/j.1365-2729.2008.00299.x. URL: https://onlinelibrary.wiley.com/doi/abs/10.1111/j.1365-2729.2008.00299.x.

[52] David Wood, Jerome S Bruner und Gail Ross. „The role of tutoring in problem solving". In: *Journal of child psychology and psychiatry* 17.2 (1976), S. 89–100.

[53] Zachtronics. *SHENZHEN I/O*. Steam Store. 2016. URL: https://store.steampowered.com/app/504210/SHENZHEN_IO/.